V Recurrent Neural Networks

Crash Course in Recurrent Neural Networks

30

Another type of neural network is dominating difficult machine learning problems involving sequences of inputs: recurrent neural networks. Recurrent neural networks have connections that have loops, adding feedback and memory to the networks over time. This memory allows this type of network to learn and generalize across sequences of inputs rather than individual patterns.

A powerful type of Recurrent Neural Network called the Long Short-Term Memory Network has been shown to be particularly effective when stacked into a deep configuration, achieving state-of-the-art results on a diverse array of problems from language translation to automatic captioning of images and videos. In this chapter, you will get a crash course in recurrent neural networks for deep learning, acquiring just enough understanding to start using LSTM networks in Python with Keras. After reading this chapter, you will know:

- ▷ The limitations of Multilayer Perceptrons that are addressed by recurrent neural networks
- ▷ The problems that must be addressed to make Recurrent Neural networks useful
- ▷ The details of the Long Short-Term Memory networks used in applied deep learning

Let's get started.

Overview

This chapter is divided into five sections; they are:

- ▷ Support For Sequences in Neural Networks
- ▷ Recurrent Neural Networks
- ▷ Long Short-Term Memory Networks

30.1 Support for Sequences in Neural Networks

Some problem types are best framed by involving either a sequence as an input or an output. For example, consider a univariate time series problem, like the price of a stock over time. This dataset can be framed as a prediction problem for a classical feedforward multilayer

perceptron network by defining a window's size (e.g., 5) and training the network to learn to make short-term predictions from the fixed-sized window of inputs.

This would work but is very limited. The window of inputs adds memory to the problem but is limited to just a fixed number of points and must be chosen with sufficient knowledge of the problem. A naive window would not capture the broader trends over minutes, hours, and days that might be relevant to making a prediction. From one prediction to the next, the network only knows about the specific inputs it is provided. Univariate time series prediction is important, but there are even more interesting problems that involve sequences. Consider the following taxonomy of sequence problems that require mapping an input to output (taken from Andrej Karpathy):

- ▷ **One-to-Many**: sequence output for image captioning
- ▷ **Many-to-One**: sequence input for sentiment classification
- ▷ **Many-to-Many**: sequence in and out for machine translation
- ▷ **Synchronized Many-to-Many**: synced sequences in and out for video classification

You can also see that a one-to-one example of an input to output would be an example of a classical feedforward neural network for a prediction task like image classification. Support for sequences as input is an important class of problem and one where deep learning has recently shown impressive results. State-of-the art results have been shown using recurrent neural networks — a type of network specifically designed for sequence problems.

30.2 Recurrent Neural Networks

Recurrent neural networks or RNNs are a special type of neural network designed for sequence problems. Given a standard feedforward multilayer perceptron network, a recurrent neural network can be thought of as the addition of loops to the architecture. For example, in a given layer, each neuron may pass its signal laterally (sideways) in addition to forward to the next layer. The output of the network may feed back as an input to the network with the next input vector. And so on.

The recurrent connections add state or memory to the network and allow it to learn broader abstractions from the input sequences. The field of recurrent neural networks is well established with popular methods. For the techniques to be effective on real problems, two major issues needed to be resolved for the network to be useful.

1. How to train the network with backpropagation
2. How to stop gradients vanishing or exploding during training

How to Train Recurrent Neural Networks

The staple technique for training feedforward neural networks is to backpropagate error and update the network weights. Backpropagation breaks down in a recurrent neural network because of the recurrent or loop connections. This was addressed with a modification of the backpropagation technique called Backpropagation Through Time or BPTT.

Instead of performing backpropagation on the recurrent network as stated, the structure of the network is unrolled, where copies of the neurons that have recurrent connections are created. For example, a single neuron with a connection to itself ($A \rightarrow A$) could be represented as two neurons with the same weight values ($A \rightarrow B$). This allows the cyclic graph of a recurrent neural network to be turned into an acyclic graph like a classic feedforward neural network, and backpropagation can be applied.

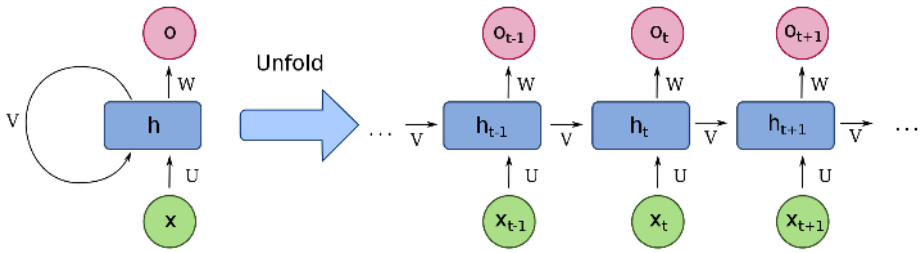

Figure 30.1: Illustration of RNN and unrolling. From Recurrent neural network *on Wikipedia*

How to Have Stable Gradients During Training

When backpropagation is used in very deep neural networks and unrolled recurrent neural networks, the gradients that are calculated to update the weights can become unstable. They can become very large numbers called exploding gradients, or very small numbers called the vanishing gradient problem. These large numbers, in turn, are used to update the weights in the network, making training unstable and the network unreliable.

This problem is alleviated in deep multilayer perceptron networks through the use of the rectifier transfer function and even more exotic but now less popular approaches of using unsupervised pre-training of layers. In recurrent neural network architectures, this problem has been alleviated using a new type of architecture called the Long Short-Term Memory Networks that allows deep recurrent networks to be trained.

30.3 Long Short-Term Memory Networks

The Long Short-Term Memory network, or LSTM network, is a recurrent neural network trained using Backpropagation Through Time and overcomes the vanishing gradient problem. As such, it can be used to create large (stacked) recurrent networks that, in turn, can be used to address difficult sequence problems in machine learning and achieve state-of-the-art results.

Instead of neurons, LSTM networks have memory blocks connected into layers. A block has components that make it smarter than a classical neuron and a memory for recent sequences. A block contains gates that manage the block's state and output. A unit operates upon an input sequence, and each gate within a unit uses the sigmoid activation function to control

whether it is triggered or not, making the change of state and addition of information flowing through the unit conditional. There are three types of gates within a memory unit:

- **Forget Gate**: conditionally decides what information to discard from the unit.
- **Input Gate**: conditionally decides which values from the input to update the memory state.
- **Output Gate**: conditionally decides what to output based on input and the memory of the unit.

Each unit is like a mini state machine where the gates of the units have weights that are learned during the training procedure. You can see how you may achieve sophisticated learning and memory from a layer of LSTMs, and it is not hard to imagine how higher-order abstractions may be layered with such multiple layers.

30.4 Further Reading

We have covered a lot of ground in this chapter. Below are some resources that you can use to go deeper into the topic of recurrent neural networks for deep learning.

Resources to learn more about recurrent neural networks and LSTMs:

Resources to learn more about Recurrent Neural Networks and LSTMs

Recurrent neural network. Wikipedia.
https://en.wikipedia.org/wiki/Recurrent_neural_network

Andrej Karpathy. *The Unreasonable Effectiveness of Recurrent Neural Networks*. May 2015.
http://karpathy.github.io/2015/05/21/rnn-effectiveness/

Backpropagation through time. Wikipedia.
https://en.wikipedia.org/wiki/Backpropagation_through_time

Aston Zhang et al. Chapter 10, "Modern Recurrent Neural Networks". In: *Dive into Deep Learning*. 2022.
http://www.d2l.ai/chapter_recurrent-modern/deep-rnn.html

Christopher Olah. *Understanding LSTM networks*. Aug. 2015.
https://colah.github.io/posts/2015-08-Understanding-LSTMs/

Long short-term memory. Wikipedia.
https://en.wikipedia.org/wiki/Long_short-term_memory

Popular tutorials for implementing LSTMs:

Recurrent Neural Networks (RNN) with Keras. TensorFlow guide.
https://www.tensorflow.org/guide/keras/rnn

Time series forecasting. TensorFlow tutorials.
https://www.tensorflow.org/tutorials/structured_data/time_series

Text generation with an RNN. TensorFlow tutorials.
https://www.tensorflow.org/text/tutorials/text_generation

Primary sources on LSTMs:

Sepp Hochreiter and Jürgen Schmidhuber. "Long Short-Term Memory". *Neural Computation*, 9(8), Nov. 1997, pp. 1735–1780. DOI: 10.1162/neco.1997.9.81735.

Felix A. Gers, Jürgen Schmidhuber, and Fred Cummins. "Learning to Forget: Continual Prediction with LSTM". *Neural Computation*, 12(10), Oct. 2000, pp. 2451–2471. DOI: 10.1162/089976600300015015.

Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. "On the difficulty of training Recurrent Neural Networks". *Proceedings of Machine Learning Reserach*, 28(3), 2013, pp. 1310–1318. https://arxiv.org/pdf/1211.5063v2.pdf

30.5 Summary

In this chapter, you discovered sequence problems and recurrent neural networks that can be used to address them.

Specifically, you learned:

- ▷ The limitations of classical feedforward neural networks and how recurrent neural networks can overcome these problems
- ▷ The practical problems in training recurrent neural networks and how they are overcame
- ▷ The Long Short-Term Memory network used to create deep recurrent neural networks

In the next chapter, you will learn some techniques on using neural network to predict time series without LSTM.

Time Series Prediction with Multilayer Perceptrons

31

Time series prediction is a difficult problem both to frame and address with machine learning. In this chapter, you will discover how to develop neural network models for time series prediction in Python using the Keras deep learning library. After reading this chapter, you will know:

- About the airline passengers univariate time series prediction problem
- How to phrase time series prediction as a regression problem and develop a neural network model for it
- How to frame time series prediction with a time lag and develop a neural network model for it

Let's get started.

Overview

This chapter is divided into five sections; they are:

- The Time Series Prediction Problem
- Multilayer Perceptron Regression
- Multilayer Perceptron Using the Window Method

31.1 The Time Series Prediction Problem

The problem you will look at in this chapter is the international airline passengers prediction problem. This is a problem where, given a year and a month, the task is to predict the number of international airline passengers in units of 1,000. The data ranges from January 1949 to December 1960, or 12 years, with 144 observations. The data is from a book by Box, Jenkins, and Reinsel. You can get it from the source code bundle of this book or download[1] the dataset in CSV format and save it with the filename `international-airline-passengers.csv`. Below is a sample of the first few lines of the file.

[1] https://raw.githubusercontent.com/jbrownlee/Datasets/master/airline-passengers.csv

```
"Month","Passengers"
"1949-01",112
"1949-02",118
"1949-03",132
"1949-04",129
```

Output 31.1: Sample output from evaluating the baseline model

You can load this dataset easily using the Pandas library. You are not interested in the date, given that each observation is separated by the same interval of one month. Therefore, when you load the dataset, you can exclude the first column. Once loaded, you can easily plot the whole dataset. The code to load and plot the dataset is listed below.

```
import pandas as pd
import matplotlib.pyplot as plt
dataset = pd.read_csv('airline-passengers.csv', usecols=[1], engine='python')
plt.plot(dataset)
plt.show()
```

Listing 31.1: Load and plot the time series dataset

You can see an upward trend in the plot. You can also see some periodicity in the dataset that probably corresponds to the northern hemisphere summer holiday period.

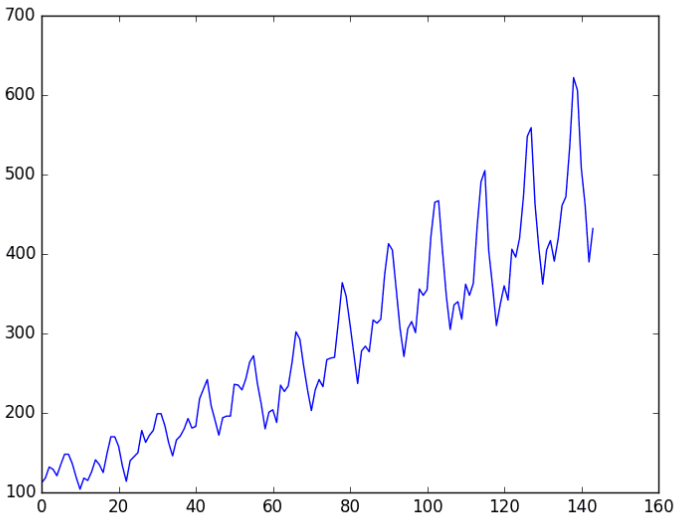

Figure 31.1: Plot of the airline passengers dataset

Let's keep things simple and work with the data as-is. Normally, it is a good idea to investigate various data preparation techniques to rescale the data and make it stationary.

31.2 Multilayer Perceptron Regression

You want to phrase the time series prediction problem as a regression problem. That is, given the number of passengers (in units of thousands) this month, what is the number of passengers

next month? You can write a simple function to convert your single column of data into a two-column dataset: The first column containing this month's (t) passenger count and the second column containing next month's ($t+1$) passenger count to be predicted. Before you get started, let's first import all the functions and classes you will need to use.

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
...
```

Listing 31.2: Import classes and functions

You can also use the code from the previous section to load the dataset as a Pandas dataframe. You can then extract the NumPy array from the dataframe and convert the integer values to floating point values, which are more suitable for modeling with a neural network.

```
...
# load the dataset
dataframe = pd.read_csv('airline-passengers.csv', usecols=[1], engine='python')
dataset = dataframe.values
dataset = dataset.astype('float32')
```

Listing 31.3: Load the time series dataset

After you model the data and estimate the skill of your model on the training dataset, you need to get an idea of the skill of the model on new unseen data. For a normal classification or regression problem, you would do this using k-fold cross-validation. With time series data, the sequence of values is important. A simple method that you can use is to split the ordered dataset into train and test datasets. The code below calculates the index of the split point and separates the data into the training datasets with 67% of the observations used to train your model, leaving the remaining 33% for testing the model.

```
...
# split into train and test sets
train_size = int(len(dataset) * 0.67)
test_size = len(dataset) - train_size
train, test = dataset[0:train_size,:], dataset[train_size:len(dataset),:]
print(len(train), len(test))
```

Listing 31.4: Split dataset into train and test

Now, you can define a function to create a new dataset as described above. The function takes two arguments: the dataset, which is a NumPy array that you want to convert into a dataset, and the `look_back`, which is the number of previous time steps to use as input variables to predict the next time period, in this case, defaulted to 1. This default will create a dataset where X is the number of passengers at a given time (t), and Y is the number of passengers at the next time ($t+1$). It can be configured, and you will look at constructing a differently shaped dataset in the next section.

```
...
# convert an array of values into a dataset matrix
def create_dataset(dataset, look_back=1):
    dataX, dataY = [], []
    for i in range(len(dataset)-look_back-1):
        a = dataset[i:(i+look_back), 0]
        dataX.append(a)
        dataY.append(dataset[i + look_back, 0])
    return np.array(dataX), np.array(dataY)
```

Listing 31.5: Function to prepare dataset for modeling

Let's take a look at the effect of this function on the first few rows of the dataset.

```
X       Y
112     118
118     132
132     129
129     121
121     135
```

Output 31.2: Sample of the prepared dataset

If you compare these first five rows to the original dataset sample listed in the previous section, you can see the $X = t$ and $Y = t+1$ pattern in the numbers. Let's use this function to prepare the train and test datasets for modeling.

```
...
# reshape into X=t and Y=t+1
look_back = 1
trainX, trainY = create_dataset(train, look_back)
testX, testY = create_dataset(test, look_back)
```

Listing 31.6: Call function to prepare dataset for modeling

We can now fit a multilayer perceptron model to the training data. We use a simple network with one input, one hidden layer with eight neurons, and an output layer. The model is fit using mean squared error, which, if you take the square root, gives you an error score in the units of the dataset. I tried a few rough parameters and settled on the configuration below, but by no means is the network listed optimized.

```
...
# create and fit Multilayer Perceptron model
model = Sequential()
model.add(Dense(8, input_shape=(look_back,), activation='relu'))
model.add(Dense(1))
model.compile(loss='mean_squared_error', optimizer='adam')
model.fit(trainX, trainY, epochs=200, batch_size=2, verbose=2)
```

Listing 31.7: Define and fit multilayer perceptron model

Once the model is fit, you can estimate the performance of the model on the train and test datasets. This will give you a point of comparison for new models.

```
...
# Estimate model performance
trainScore = model.evaluate(trainX, trainY, verbose=0)
print('Train Score: %.2f MSE (%.2f RMSE)' % (trainScore, math.sqrt(trainScore)))
testScore = model.evaluate(testX, testY, verbose=0)
print('Test Score: %.2f MSE (%.2f RMSE)' % (testScore, math.sqrt(testScore)))
```

Listing 31.8: Evaluate the fit model

Finally, you can generate predictions using the model for both the train and test dataset to get a visual indication of the skill of the model. Because of how the dataset was prepared, you must shift the predictions to align on the x-axis with the original dataset. Once prepared, the data is plotted, showing the original dataset in blue, the predictions for the training dataset in green, and the predictions on the unseen test dataset in red.

```
...
# generate predictions for training
trainPredict = model.predict(trainX)
testPredict = model.predict(testX)

# shift train predictions for plotting
trainPredictPlot = np.empty_like(dataset)
trainPredictPlot[:, :] = np.nan
trainPredictPlot[look_back:len(trainPredict)+look_back, :] = trainPredict

# shift test predictions for plotting
testPredictPlot = np.empty_like(dataset)
testPredictPlot[:, :] = np.nan
testPredictPlot[len(trainPredict)+(look_back*2)+1:len(dataset)-1, :] = testPredict

# plot baseline and predictions
plt.plot(dataset)
plt.plot(trainPredictPlot)
plt.plot(testPredictPlot)
plt.show()
```

Listing 31.9: Generate and plot predictions

Tying this all together, the complete example is listed below.

```
# Multilayer Perceptron to Predict International Airline Passengers (t+1, given t)
import numpy as np
import matplotlib.pyplot as plt
from pandas import read_csv
import math
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

# convert an array of values into a dataset matrix
def create_dataset(dataset, look_back=1):
    dataX, dataY = [], []
    for i in range(len(dataset)-look_back-1):
        a = dataset[i:(i+look_back), 0]
```

```
        dataX.append(a)
        dataY.append(dataset[i + look_back, 0])
    return np.array(dataX), np.array(dataY)

# load the dataset
dataframe = read_csv('airline-passengers.csv', usecols=[1], engine='python')
dataset = dataframe.values
dataset = dataset.astype('float32')
# split into train and test sets
train_size = int(len(dataset) * 0.67)
test_size = len(dataset) - train_size
train, test = dataset[0:train_size,:], dataset[train_size:len(dataset),:]
# reshape into X=t and Y=t+1
look_back = 1
trainX, trainY = create_dataset(train, look_back)
testX, testY = create_dataset(test, look_back)
# create and fit Multilayer Perceptron model
model = Sequential()
model.add(Dense(8, input_shape=(look_back,), activation='relu'))
model.add(Dense(1))
model.compile(loss='mean_squared_error', optimizer='adam')
model.fit(trainX, trainY, epochs=200, batch_size=2, verbose=2)
# Estimate model performance
trainScore = model.evaluate(trainX, trainY, verbose=0)
print('Train Score: %.2f MSE (%.2f RMSE)' % (trainScore, math.sqrt(trainScore)))
testScore = model.evaluate(testX, testY, verbose=0)
print('Test Score: %.2f MSE (%.2f RMSE)' % (testScore, math.sqrt(testScore)))
# generate predictions for training
trainPredict = model.predict(trainX)
testPredict = model.predict(testX)
# shift train predictions for plotting
trainPredictPlot = np.empty_like(dataset)
trainPredictPlot[:, :] = np.nan
trainPredictPlot[look_back:len(trainPredict)+look_back, :] = trainPredict
# shift test predictions for plotting
testPredictPlot = np.empty_like(dataset)
testPredictPlot[:, :] = np.nan
testPredictPlot[len(trainPredict)+(look_back*2)+1:len(dataset)-1, :] = testPredict
# plot baseline and predictions
plt.plot(dataset, 'b')
plt.plot(trainPredictPlot, 'g')
plt.plot(testPredictPlot, 'r')
plt.show()
```

Listing 31.10: Multilayer perceptron model for the $t + 1$ model

Running the example reports the model performance.

Note: Your results may vary given the stochastic nature of the algorithm or evaluation procedure, or differences in numerical precision. Consider running the example a few times and compare the average outcome.

Taking the square root of the performance estimates, you can see that the model has an average error of 22 passengers (in thousands) on the training dataset and 46 passengers (in thousands) on the test dataset.

```
...
Epoch 395/400
46/46 - 0s - loss: 513.2617 - 13ms/epoch - 275us/step
Epoch 396/400
46/46 - 0s - loss: 494.1868 - 12ms/epoch - 268us/step
Epoch 397/400
46/46 - 0s - loss: 483.3908 - 12ms/epoch - 268us/step
Epoch 398/400
46/46 - 0s - loss: 501.8111 - 13ms/epoch - 281us/step
Epoch 399/400
46/46 - 0s - loss: 523.2578 - 13ms/epoch - 280us/step
Epoch 400/400
46/46 - 0s - loss: 513.7587 - 12ms/epoch - 263us/step
Train Score: 487.39 MSE (22.08 RMSE)
Test Score: 2070.68 MSE (45.50 RMSE)
```

Output 31.3: Sample output of the multilayer perceptron model for the $t+1$ model

From the plot, you can see that the model did a pretty poor job of fitting both the training and the test datasets. It basically predicted the same input value as the output.

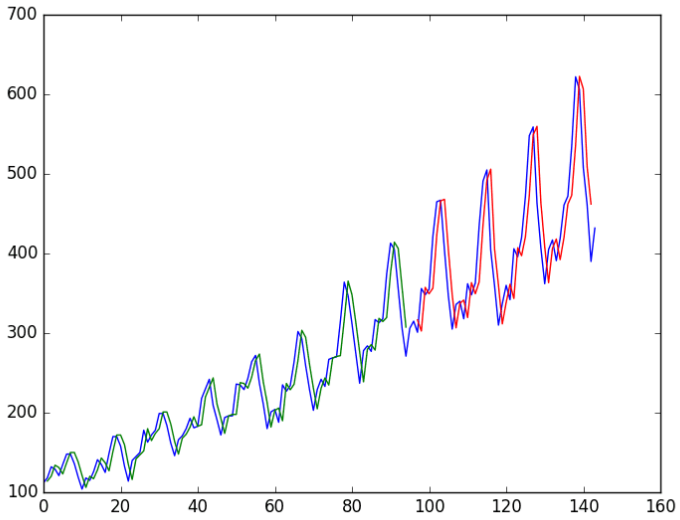

Figure 31.2: Naive time series predictions with neural network. Blue=Whole dataset; Green=Training; Red=Predictions

31.3 Multilayer Perceptron Using the Window Method

You can also phrase the problem so that multiple recent time steps can be used to make the prediction for the next time step. This is called the window method, and the size of the

window is a parameter that can be tuned for each problem. For example, given the current time (t) to predict the value at the next time in the sequence ($t+1$), you can use the current time (t) as well as the two prior times ($t-1$ and $t-2$). When phrased as a regression problem, the input variables are $t-2$, $t-1$, t and the output variable is $t+1$.

The `create_dataset()` function used in the previous section allows you to create this formulation of the time series problem by increasing the `look_back` argument from 1 to 3. A sample of the dataset with this formulation looks as follows:

```
X1     X2     X3     Y
112    118    132    129
118    132    129    121
132    129    121    135
129    121    135    148
121    135    148    148
```

Output 31.4: Sample dataset of the window formulation of the problem

You can re-run the example in the previous section with the larger window size. You will increase the network capacity to handle the additional information. The first hidden layer is increased to 14 neurons, and a second hidden layer is added with eight neurons. The number of epochs is also increased to 400.

The whole code listing with just the window size change is listed below for completeness.

```
# Multilayer Perceptron to Predict International Airline Passengers (t+1, given
# t, t-1, t-2)
import numpy as np
import matplotlib.pyplot as plt
from pandas import read_csv
import math
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

# convert an array of values into a dataset matrix
def create_dataset(dataset, look_back=1):
    dataX, dataY = [], []
    for i in range(len(dataset)-look_back-1):
        a = dataset[i:(i+look_back), 0]
        dataX.append(a)
        dataY.append(dataset[i + look_back, 0])
    return np.array(dataX), np.array(dataY)

# load the dataset
dataframe = read_csv('airline-passengers.csv', usecols=[1], engine='python')
dataset = dataframe.values
dataset = dataset.astype('float32')
# split into train and test sets
train_size = int(len(dataset) * 0.67)
test_size = len(dataset) - train_size
train, test = dataset[0:train_size,:], dataset[train_size:len(dataset),:]
# reshape dataset
look_back = 3
```

```
trainX, trainY = create_dataset(train, look_back)
testX, testY = create_dataset(test, look_back)
# create and fit Multilayer Perceptron model
model = Sequential()
model.add(Dense(12, input_shape=(look_back,), activation='relu'))
model.add(Dense(8, activation='relu'))
model.add(Dense(1))
model.compile(loss='mean_squared_error', optimizer='adam')
model.fit(trainX, trainY, epochs=400, batch_size=2, verbose=2)
# Estimate model performance
trainScore = model.evaluate(trainX, trainY, verbose=0)
print('Train Score: %.2f MSE (%.2f RMSE)' % (trainScore, math.sqrt(trainScore)))
testScore = model.evaluate(testX, testY, verbose=0)
print('Test Score: %.2f MSE (%.2f RMSE)' % (testScore, math.sqrt(testScore)))
# generate predictions for training
trainPredict = model.predict(trainX)
testPredict = model.predict(testX)
# shift train predictions for plotting
trainPredictPlot = np.empty_like(dataset)
trainPredictPlot[:, :] = np.nan
trainPredictPlot[look_back:len(trainPredict)+look_back, :] = trainPredict
# shift test predictions for plotting
testPredictPlot = np.empty_like(dataset)
testPredictPlot[:, :] = np.nan
testPredictPlot[len(trainPredict)+(look_back*2)+1:len(dataset)-1, :] = testPredict
# plot baseline and predictions
plt.plot(dataset, 'b')
plt.plot(trainPredictPlot, 'g')
plt.plot(testPredictPlot, 'r')
plt.show()
```

Listing 31.11: Multilayer perceptron model for the window method

Note: Your results may vary given the stochastic nature of the algorithm or evaluation procedure, or differences in numerical precision. Consider running the example a few times and compare the average outcome.

Running the example provides the following output.

```
Epoch 395/400
46/46 - 0s - loss: 419.0309 - 14ms/epoch - 294us/step
Epoch 396/400
46/46 - 0s - loss: 429.3398 - 14ms/epoch - 300us/step
Epoch 397/400
46/46 - 0s - loss: 412.2588 - 14ms/epoch - 298us/step
Epoch 398/400
46/46 - 0s - loss: 424.6126 - 13ms/epoch - 292us/step
Epoch 399/400
46/46 - 0s - loss: 429.6443 - 14ms/epoch - 296us/step
Epoch 400/400
46/46 - 0s - loss: 419.9067 - 14ms/epoch - 301us/step
```

```
Train Score: 393.07 MSE (19.83 RMSE)
Test Score: 1833.35 MSE (42.82 RMSE)
```

Output 31.5: Sample output of the multilayer perceptron model for the window model

You can see that the error was not significantly reduced compared to that of the previous section. Looking at the graph, you can see more structure in the predictions. Again, the window size and the network architecture were not tuned; this is just a demonstration of how to frame a prediction problem. Taking the square root of the performance scores, you can see the average error on the training dataset was 20 passengers (in thousands per month), and the average error on the unseen test set was 43 passengers (in thousands per month).

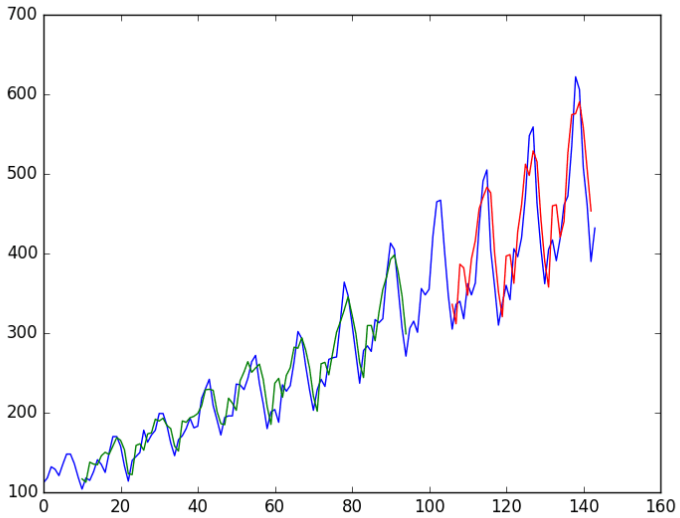

Figure 31.3: Window method for time series predictions with neural networks. Blue=Whole dataset; Green=Training; Red=Predictions

31.4 Further Reading

This section provides more resources on the topic if you are looking to go deeper.

Books

G. E. P. Box, G. M. Jenkins, and G. C. Reinsel. *Time Series Analysis, Forecasting and Control*. 3rd ed. Holden-Day, 1976.

31.5 Summary

In this chapter, you discovered how to develop a neural network model for a time series prediction problem using the Keras deep learning library. After working through this chapter, you now know:

- ▷ About the international airline passenger prediction time series dataset
- ▷ How to frame time series prediction problems as regression problems and develop a neural network model
- ▷ How use the window approach to frame a time series prediction problem and develop a neural network model

In the next chapter, you will see how LSTM can be used for the same problem.

Time Series Prediction with LSTM Networks

32

Time series prediction problems are a difficult type of predictive modeling problem. Unlike regression predictive modeling, time series also adds the complexity of a sequence dependence among the input variables. A powerful type of neural network designed to handle sequence dependence is called a recurrent neural network. The Long Short-Term Memory network or LSTM network is a type of recurrent neural network used in deep learning because very large architectures can be successfully trained.

In this chapter, you will discover how to develop LSTM networks in Python using the Keras deep learning library to address a demonstration time series prediction problem. After completing this chapter, you will know how to implement and develop LSTM networks for your own time series prediction problems and other more general sequence problems. You will know:

- How to develop LSTM networks for a time series prediction problem framed as regression
- How to develop LSTM networks for a time series prediction problem using a window, for both features and time steps
- How to develop and make predictions using LSTM networks that maintain state (memory) across very long sequences

In this chapter, we will develop a number of LSTMs for a standard time series prediction problem. The problem and the chosen configuration for the LSTM networks are for demonstration purposes only. They are not optimized. These examples will show exactly how you can develop your own differently structured LSTM networks for time series predictive modeling problems.

Let's get started.

Overview

This chapter is divided into six sections; they are:

- Problem Description
- LSTM Network for Regression

- ▷ LSTM for Regression Using the Window Method
- ▷ LSTM for Regression with Time Steps
- ▷ LSTM with Memory Between Batches
- ▷ Stacked LSTMs with Memory Between Batches

32.1 Problem Description

The problem you will look at in this chapter is the international airline passengers prediction problem, described in Section 31.1. This is a problem where, given a year and a month, the task is to predict the number of international airline passengers in units of 1,000. The data ranges from January 1949 to December 1960, or 12 years, with 144 observations.

We can phrase the problem as a regression problem, as was done in the previous chapter. That is, given the number of passengers (in units of thousands) this month, what is the number of passengers next month. This example will reuse the same data loading and preparation from the previous chapter, specifically the use of the `create_dataset()` function.

We will use LSTM network, described in Section 30.3, for the prediction.

32.2 LSTM Network for Regression

You can write a simple function to convert the single column of data into a two-column dataset: the first column containing this month's (t) passenger count and the second column containing next month's ($t + 1$) passenger count, to be predicted.

Before you start, let's first import all the functions and classes you will use. This assumes a working Python environment with the Keras deep learning library installed.

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.layers import LSTM
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error
```

Listing 32.1: Importing classes and functions

Before you do anything, it is a good idea to fix the random number seed to ensure your results are reproducible.

```
# fix random seed for reproducibility
tf.random.set_seed(7)
```

Listing 32.2: Fix the random seed for reproducibility

You can also use the code from the previous chapter to load the dataset as a Pandas dataframe. You can then extract the NumPy array from the dataframe and convert the integer values to floating point values, which are more suitable for modeling with a neural network.

```
# load the dataset
dataframe = pd.read_csv('airline-passengers.csv', usecols=[1], engine='python')
dataset = dataframe.values
dataset = dataset.astype('float32')
```

Listing 32.3: Load dataset as a Pandas dataframe

LSTMs are sensitive to the scale of the input data, specifically when the sigmoid (default) or tanh activation functions are used. It can be a good practice to rescale the data to the range of 0-to-1, also called normalizing. You can easily normalize the dataset using the `MinMaxScaler` preprocessing class from the scikit-learn library.

```
# normalize the dataset
scaler = MinMaxScaler(feature_range=(0, 1))
dataset = scaler.fit_transform(dataset)
```

Listing 32.4: Normalize the dataset with MinMaxScaler

After you model the data and estimate the skill of your model on the training dataset, you need to get an idea of the skill of the model on new unseen data. For a normal classification or regression problem, you would do this using cross-validation.

With time series data, the sequence of values is important. A simple method that you can use is to split the ordered dataset into train and test datasets. The code below calculates the index of the split point and separates the data into the training datasets, with 67% of the observations used to train the model, leaving the remaining 33% for testing the model.

```
# split into train and test sets
train_size = int(len(dataset) * 0.67)
test_size = len(dataset) - train_size
train, test = dataset[0:train_size,:], dataset[train_size:len(dataset),:]
print(len(train), len(test))
```

Listing 32.5: Split the data into training and test sets

Now, you can define a function to create a new dataset, as described above. The function takes two arguments: the `dataset`, which is a NumPy array you want to convert into a dataset, and the `look_back`, which is the number of previous time steps to use as input variables to predict the next time period — in this case, defaulted to 1. This default will create a dataset where X is the number of passengers at a given time (t), and Y is the number of passengers at the next time $(t + 1)$. It can be configured by constructing a differently shaped dataset in the next section.

```
# convert an array of values into a dataset matrix
def create_dataset(dataset, look_back=1):
    dataX, dataY = [], []
    for i in range(len(dataset)-look_back-1):
```

```
        a = dataset[i:(i+look_back), 0]
        dataX.append(a)
        dataY.append(dataset[i + look_back, 0])
    return np.array(dataX), np.array(dataY)
```

Listing 32.6: Helper function to convert a time series into a dataset matrix

Let's take a look at the effect of this function on the first rows of the dataset (shown in the unnormalized form for clarity).

```
X          Y
112        118
118        132
132        129
129        121
121        135
```

Output 32.1: Sample dataset as prepared

If you compare these first five rows to the original dataset sample listed in the previous section, you can see the $X = t$ and $Y = t + 1$ pattern in the numbers.

Let's use this function to prepare the train and test datasets for modeling.

```
# reshape into X=t and Y=t+1
look_back = 1
trainX, trainY = create_dataset(train, look_back)
testX, testY = create_dataset(test, look_back)
```

Listing 32.7: Prepare the train and test datasets

The LSTM network expects the input data (X) to be provided with a specific array structure in the form of [*samples, time steps, features*]. Currently, the data is in the form of [*samples, features*], and you are framing the problem as one time step for each sample. You can transform the prepared train and test input data into the expected structure using `numpy.reshape()` as follows:

```
# reshape input to be [samples, time steps, features]
trainX = np.reshape(trainX, (trainX.shape[0], 1, trainX.shape[1]))
testX = np.reshape(testX, (testX.shape[0], 1, testX.shape[1]))
```

Listing 32.8: Reshape the prepared dataset for the LSTM layers

You are now ready to design and fit your LSTM network for this problem. The network has a visible layer with 1 input, a hidden layer with 4 LSTM blocks or neurons, and an output layer that makes a single value prediction. The default sigmoid activation function is used for the LSTM memory blocks. The network is trained for 100 epochs, and a batch size of 1 is used.

```
# create and fit the LSTM network
model = Sequential()
model.add(LSTM(4, input_shape=(1, look_back)))
model.add(Dense(1))
```

```
model.compile(loss='mean_squared_error', optimizer='adam')
model.fit(trainX, trainY, epochs=100, batch_size=1, verbose=2)
```

Listing 32.9: Define and fit the LSTM network

Once the model is fit, you can estimate the performance of the model on the train and test datasets. This will give you a point of comparison for new models. Note that you will invert the predictions before calculating error scores to ensure that performance is reported in the same units as the original data (thousands of passengers per month).

```
# make predictions
trainPredict = model.predict(trainX)
testPredict = model.predict(testX)
# invert predictions
trainPredict = scaler.inverse_transform(trainPredict)
trainY = scaler.inverse_transform([trainY])
testPredict = scaler.inverse_transform(testPredict)
testY = scaler.inverse_transform([testY])
# calculate root mean squared error
trainScore = np.sqrt(mean_squared_error(trainY[0], trainPredict[:,0]))
print('Train Score: %.2f RMSE' % (trainScore))
testScore = np.sqrt(mean_squared_error(testY[0], testPredict[:,0]))
print('Test Score: %.2f RMSE' % (testScore))
```

Listing 32.10: Estimate the RMSE of prediction using LSTM network

Finally, you can generate predictions using the model for both the train and test dataset to get a visual indication of the skill of the model. Because of how the dataset was prepared, you must shift the predictions so that they align on the x-axis with the original dataset. Once prepared, the data is plotted, showing the original dataset in blue, the predictions for the training dataset in green, and the predictions on the unseen test dataset in red.

```
# shift train predictions for plotting
trainPredictPlot = np.empty_like(dataset)
trainPredictPlot[:, :] = np.nan
trainPredictPlot[look_back:len(trainPredict)+look_back, :] = trainPredict
# shift test predictions for plotting
testPredictPlot = np.empty_like(dataset)
testPredictPlot[:, :] = np.nan
testPredictPlot[len(trainPredict)+(look_back*2)+1:len(dataset)-1, :] = testPredict
# plot baseline and predictions
plt.plot(scaler.inverse_transform(dataset))
plt.plot(trainPredictPlot)
plt.plot(testPredictPlot)
plt.show()
```

Listing 32.11: Align predictions and plot

You can see that the model fit both the training and the test datasets.

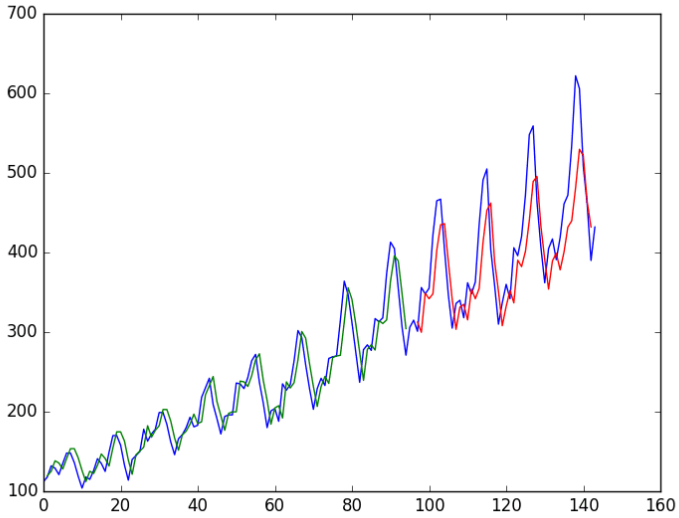

Figure 32.1: LSTM trained on regression formulation of passenger prediction problem

For completeness, below is the entire code example.

```
# LSTM for international airline passengers problem with regression framing
import numpy as np
import matplotlib.pyplot as plt
from pandas import read_csv
import math
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.layers import LSTM
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error
# convert an array of values into a dataset matrix
def create_dataset(dataset, look_back=1):
    dataX, dataY = [], []
    for i in range(len(dataset)-look_back-1):
        a = dataset[i:(i+look_back), 0]
        dataX.append(a)
        dataY.append(dataset[i + look_back, 0])
    return np.array(dataX), np.array(dataY)
# fix random seed for reproducibility
tf.random.set_seed(7)
# load the dataset
dataframe = read_csv('airline-passengers.csv', usecols=[1], engine='python')
dataset = dataframe.values
dataset = dataset.astype('float32')
# normalize the dataset
scaler = MinMaxScaler(feature_range=(0, 1))
dataset = scaler.fit_transform(dataset)
# split into train and test sets
train_size = int(len(dataset) * 0.67)
test_size = len(dataset) - train_size
```

```
train, test = dataset[0:train_size,:], dataset[train_size:len(dataset),:]
# reshape into X=t and Y=t+1
look_back = 1
trainX, trainY = create_dataset(train, look_back)
testX, testY = create_dataset(test, look_back)
# reshape input to be [samples, time steps, features]
trainX = np.reshape(trainX, (trainX.shape[0], 1, trainX.shape[1]))
testX = np.reshape(testX, (testX.shape[0], 1, testX.shape[1]))
# create and fit the LSTM network
model = Sequential()
model.add(LSTM(4, input_shape=(1, look_back)))
model.add(Dense(1))
model.compile(loss='mean_squared_error', optimizer='adam')
model.fit(trainX, trainY, epochs=100, batch_size=1, verbose=2)
# make predictions
trainPredict = model.predict(trainX)
testPredict = model.predict(testX)
# invert predictions
trainPredict = scaler.inverse_transform(trainPredict)
trainY = scaler.inverse_transform([trainY])
testPredict = scaler.inverse_transform(testPredict)
testY = scaler.inverse_transform([testY])
# calculate root mean squared error
trainScore = np.sqrt(mean_squared_error(trainY[0], trainPredict[:,0]))
print('Train Score: %.2f RMSE' % (trainScore))
testScore = np.sqrt(mean_squared_error(testY[0], testPredict[:,0]))
print('Test Score: %.2f RMSE' % (testScore))
# shift train predictions for plotting
trainPredictPlot = np.empty_like(dataset)
trainPredictPlot[:, :] = np.nan
trainPredictPlot[look_back:len(trainPredict)+look_back, :] = trainPredict
# shift test predictions for plotting
testPredictPlot = np.empty_like(dataset)
testPredictPlot[:, :] = np.nan
testPredictPlot[len(trainPredict)+(look_back*2)+1:len(dataset)-1, :] = testPredict
# plot baseline and predictions
plt.plot(scaler.inverse_transform(dataset))
plt.plot(trainPredictPlot)
plt.plot(testPredictPlot)
plt.show()
```

Listing 32.12: LSTM model on the $t + 1$ problem

Note: Your results may vary given the stochastic nature of the algorithm or evaluation procedure, or differences in numerical precision. Consider running the example a few times and compare the average outcome.

Running the example produces the following output.

```
...
Epoch 95/100
94/94 - 0s - loss: 0.0021 - 37ms/epoch - 391us/step
Epoch 96/100
94/94 - 0s - loss: 0.0020 - 37ms/epoch - 398us/step
Epoch 97/100
94/94 - 0s - loss: 0.0020 - 37ms/epoch - 396us/step
Epoch 98/100
94/94 - 0s - loss: 0.0020 - 37ms/epoch - 391us/step
Epoch 99/100
94/94 - 0s - loss: 0.0020 - 37ms/epoch - 394us/step
Epoch 100/100
94/94 - 0s - loss: 0.0020 - 36ms/epoch - 382us/step
3/3 [==============================] - 0s 490us/step
2/2 [==============================] - 0s 461us/step
Train Score: 22.68 RMSE
Test Score: 49.34 RMSE
```

Output 32.2: Sample output from the LSTM model on the $t + 1$ problem

You can see that the model has an average error of about 23 passengers (in thousands) on the training dataset and about 49 passengers (in thousands) on the test dataset. Not that bad.

32.3 LSTM for Regression Using the Window Method

You can also phrase the problem so that multiple, recent time steps can be used to make the prediction for the next time step. This is called a window, and the size of the window is a parameter that can be tuned for each problem. For example, given the current time (t) to predict the value at the next time in the sequence ($t+1$), you can use the current time (t), as well as the two prior times ($t-1$ and $t-2$) as input variables. When phrased as a regression problem, the input variables are $t-2$, $t-1$, and t, and the output variable is $t+1$. In this example, a window of three time steps is used. But the size of a window is a hyperparameter that you can adjust, for example, using grid search.

The `create_dataset()` function created in the previous section allows you to create this formulation of the time series problem by increasing the `look_back` argument from 1 to 3. A sample of the dataset with this formulation is as follows:

```
X1    X2    X3    Y
112   118   132   129
118   132   129   121
132   129   121   135
129   121   135   148
121   135   148   148
```

Output 32.3: Sample dataset of the window formulation of the problem

We can re-run the example in the previous section with the larger window size. The whole code listing with just the window size change is listed below for completeness.

```
# LSTM for international airline passengers problem with window regression framing
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from pandas import read_csv
from keras.models import Sequential
from keras.layers import Dense
from keras.layers import LSTM
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error
# convert an array of values into a dataset matrix
def create_dataset(dataset, look_back=1):
    dataX, dataY = [], []
    for i in range(len(dataset)-look_back-1):
        a = dataset[i:(i+look_back), 0]
        dataX.append(a)
        dataY.append(dataset[i + look_back, 0])
    return np.array(dataX), np.array(dataY)
# fix random seed for reproducibility
tf.random.set_seed(7)
# load the dataset
dataframe = read_csv('airline-passengers.csv', usecols=[1], engine='python')
dataset = dataframe.values
dataset = dataset.astype('float32')
# normalize the dataset
scaler = MinMaxScaler(feature_range=(0, 1))
dataset = scaler.fit_transform(dataset)
# split into train and test sets
train_size = int(len(dataset) * 0.67)
test_size = len(dataset) - train_size
train, test = dataset[0:train_size,:], dataset[train_size:len(dataset),:]
# reshape into X=t and Y=t+1
look_back = 3
trainX, trainY = create_dataset(train, look_back)
testX, testY = create_dataset(test, look_back)
# reshape input to be [samples, time steps, features]
trainX = np.reshape(trainX, (trainX.shape[0], 1, trainX.shape[1]))
testX = np.reshape(testX, (testX.shape[0], 1, testX.shape[1]))
# create and fit the LSTM network
model = Sequential()
model.add(LSTM(4, input_shape=(1, look_back)))
model.add(Dense(1))
model.compile(loss='mean_squared_error', optimizer='adam')
model.fit(trainX, trainY, epochs=100, batch_size=1, verbose=2)
# make predictions
trainPredict = model.predict(trainX)
testPredict = model.predict(testX)
# invert predictions
trainPredict = scaler.inverse_transform(trainPredict)
trainY = scaler.inverse_transform([trainY])
testPredict = scaler.inverse_transform(testPredict)
testY = scaler.inverse_transform([testY])
# calculate root mean squared error
trainScore = np.sqrt(mean_squared_error(trainY[0], trainPredict[:,0]))
```

```
print('Train Score: %.2f RMSE' % (trainScore))
testScore = np.sqrt(mean_squared_error(testY[0], testPredict[:,0]))
print('Test Score: %.2f RMSE' % (testScore))
# shift train predictions for plotting
trainPredictPlot = np.empty_like(dataset)
trainPredictPlot[:, :] = np.nan
trainPredictPlot[look_back:len(trainPredict)+look_back, :] = trainPredict
# shift test predictions for plotting
testPredictPlot = np.empty_like(dataset)
testPredictPlot[:, :] = np.nan
testPredictPlot[len(trainPredict)+(look_back*2)+1:len(dataset)-1, :] = testPredict
# plot baseline and predictions
plt.plot(scaler.inverse_transform(dataset))
plt.plot(trainPredictPlot)
plt.plot(testPredictPlot)
plt.show()
```

Listing 32.13: LSTM for the window model

Running the example provides the following output:

```
Epoch 95/100
92/92 - 0s - loss: 0.0023 - 35ms/epoch - 384us/step
Epoch 96/100
92/92 - 0s - loss: 0.0023 - 36ms/epoch - 389us/step
Epoch 97/100
92/92 - 0s - loss: 0.0024 - 37ms/epoch - 404us/step
Epoch 98/100
92/92 - 0s - loss: 0.0023 - 36ms/epoch - 392us/step
Epoch 99/100
92/92 - 0s - loss: 0.0022 - 36ms/epoch - 389us/step
Epoch 100/100
92/92 - 0s - loss: 0.0022 - 35ms/epoch - 384us/step
3/3 [==============================] - 0s 514us/step
2/2 [==============================] - 0s 533us/step
Train Score: 24.86 RMSE
Test Score: 70.48 RMSE
```

Output 32.4: Sample output from the LSTM model on the window problem

You can see that the error was increased slightly compared to that of the previous section. The window size and the network architecture were not tuned: This is just a demonstration of how to frame a prediction problem.

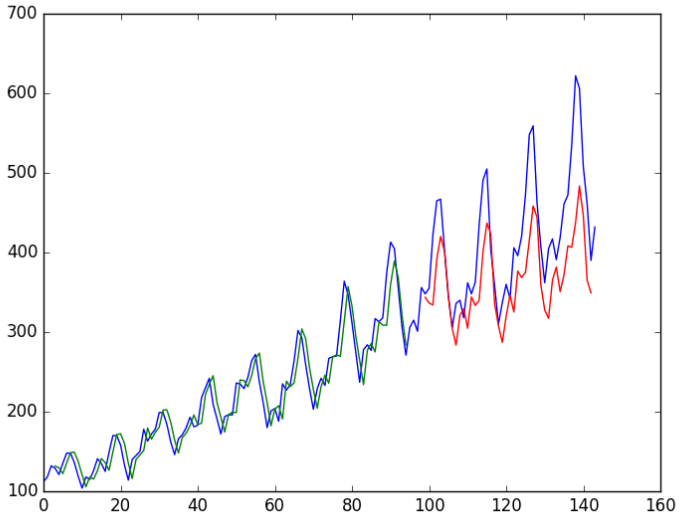

Figure 32.2: LSTM trained on window method formulation of passenger prediction problem

32.4 LSTM for Regression with Time Steps

You may have noticed that the data preparation for the LSTM network includes time steps. Some sequence problems may have a varied number of time steps per sample. For example, you may have measurements of a physical machine leading up to the point of failure or a point of surge. Each incident would be a sample of observations that lead up to the event, which would be the time steps, and the variables observed would be the features. Time steps provide another way to phrase your time series problem. Like above in the window example, you can take prior time steps in your time series as inputs to predict the output at the next time step.

Instead of phrasing the past observations as separate input features, you can use them as time steps of the one input feature, which is indeed a more accurate framing of the problem. You can do this using the same data representation as in the previous window-based example, except when you reshape the data, you set the columns to be the time steps dimension and change the features dimension back to 1. For example:

```
# reshape input to be [samples, time steps, features]
trainX = np.reshape(trainX, (trainX.shape[0], trainX.shape[1], 1))
testX = np.reshape(testX, (testX.shape[0], testX.shape[1], 1))
```

Listing 32.14: Reshape the prepared dataset for the LSTM layers with time steps

The entire code listing is provided below for completeness.

```
# LSTM for international airline passengers problem with time step regression framing
import numpy as np
import matplotlib.pyplot as plt
from pandas import read_csv
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.layers import LSTM
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error
# convert an array of values into a dataset matrix
def create_dataset(dataset, look_back=1):
    dataX, dataY = [], []
    for i in range(len(dataset)-look_back-1):
        a = dataset[i:(i+look_back), 0]
        dataX.append(a)
        dataY.append(dataset[i + look_back, 0])
    return np.array(dataX), np.array(dataY)
# fix random seed for reproducibility
tf.random.set_seed(7)
# load the dataset
dataframe = read_csv('airline-passengers.csv', usecols=[1], engine='python')
dataset = dataframe.values
dataset = dataset.astype('float32')
# normalize the dataset
scaler = MinMaxScaler(feature_range=(0, 1))
dataset = scaler.fit_transform(dataset)
# split into train and test sets
train_size = int(len(dataset) * 0.67)
test_size = len(dataset) - train_size
train, test = dataset[0:train_size,:], dataset[train_size:len(dataset),:]
# reshape into X=t and Y=t+1
look_back = 3
trainX, trainY = create_dataset(train, look_back)
testX, testY = create_dataset(test, look_back)
# reshape input to be [samples, time steps, features]
trainX = np.reshape(trainX, (trainX.shape[0], trainX.shape[1], 1))
testX = np.reshape(testX, (testX.shape[0], testX.shape[1], 1))
# create and fit the LSTM network
model = Sequential()
model.add(LSTM(4, input_shape=(look_back, 1)))
model.add(Dense(1))
model.compile(loss='mean_squared_error', optimizer='adam')
model.fit(trainX, trainY, epochs=100, batch_size=1, verbose=2)
# make predictions
trainPredict = model.predict(trainX)
testPredict = model.predict(testX)
# invert predictions
trainPredict = scaler.inverse_transform(trainPredict)
trainY = scaler.inverse_transform([trainY])
testPredict = scaler.inverse_transform(testPredict)
testY = scaler.inverse_transform([testY])
# calculate root mean squared error
trainScore = np.sqrt(mean_squared_error(trainY[0], trainPredict[:,0]))
```

```
print('Train Score: %.2f RMSE' % (trainScore))
testScore = np.sqrt(mean_squared_error(testY[0], testPredict[:,0]))
print('Test Score: %.2f RMSE' % (testScore))
# shift train predictions for plotting
trainPredictPlot = np.empty_like(dataset)
trainPredictPlot[:, :] = np.nan
trainPredictPlot[look_back:len(trainPredict)+look_back, :] = trainPredict
# shift test predictions for plotting
testPredictPlot = np.empty_like(dataset)
testPredictPlot[:, :] = np.nan
testPredictPlot[len(trainPredict)+(look_back*2)+1:len(dataset)-1, :] = testPredict
# plot baseline and predictions
plt.plot(scaler.inverse_transform(dataset))
plt.plot(trainPredictPlot)
plt.plot(testPredictPlot)
plt.show()
```

Listing 32.15: LSTM for the time steps model

Running the example provides the following output:

```
...
Epoch 95/100
92/92 - 0s - loss: 0.0023 - 45ms/epoch - 484us/step
Epoch 96/100
92/92 - 0s - loss: 0.0023 - 45ms/epoch - 486us/step
Epoch 97/100
92/92 - 0s - loss: 0.0024 - 44ms/epoch - 479us/step
Epoch 98/100
92/92 - 0s - loss: 0.0022 - 45ms/epoch - 489us/step
Epoch 99/100
92/92 - 0s - loss: 0.0022 - 45ms/epoch - 485us/step
Epoch 100/100
92/92 - 0s - loss: 0.0021 - 45ms/epoch - 490us/step
3/3 [==============================] - 0s 635us/step
2/2 [==============================] - 0s 616us/step
Train Score: 24.84 RMSE
Test Score: 60.98 RMSE
```

Output 32.5: Sample output from the LSTM model on the time step problem

You can see that the results are slightly better than the previous example. However, the structure of the input data makes a lot more sense.

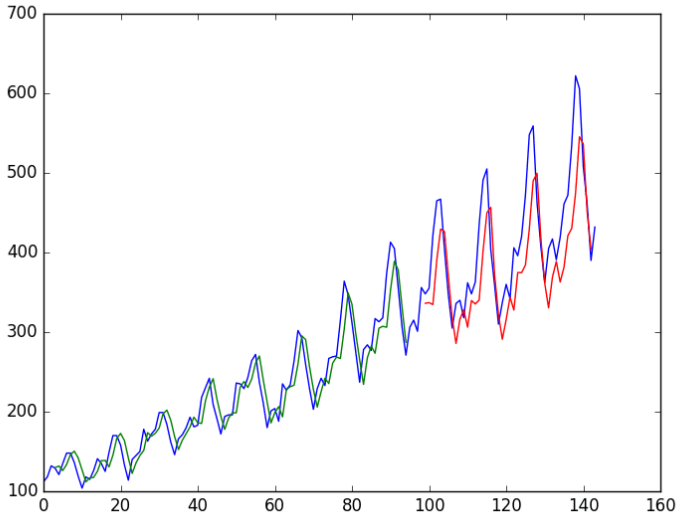

Figure 32.3: LSTM trained on time step formulation of passenger prediction problem

32.5 LSTM with Memory Between Batches

The LSTM network has memory capable of remembering across long sequences. Normally, the state within the network is reset after each training batch when fitting the model, as well as each call to `model.predict()` or `model.evaluate()`. You can gain finer control over when the internal state of the LSTM network is cleared in Keras by making the LSTM layer *stateful*. This means it can build a state over the entire training sequence and even maintain that state if needed to make predictions.

It requires that the training data not be shuffled when fitting the network. It also requires explicit resetting of the network state after each exposure to the training data (epoch) by calls to `model.reset_states()`. This means that you must create your own outer loop of epochs and within each epoch call `model.fit()` and `model.reset_states()`. For example:

```
for i in range(100):
    model.fit(trainX, trainY, epochs=1, batch_size=batch_size, verbose=2, shuffle=False)
    model.reset_states()
```

Listing 32.16: Manually reset LSTM state between epochs

Finally, when the LSTM layer is constructed, the `stateful` parameter must be set to `True`. Instead of specifying the input dimensions, you must hard code the number of samples in a batch, the number of time steps in a sample, and the number of features in a time step by setting the `batch_input_shape` parameter. For example:

```
model.add(LSTM(4, batch_input_shape=(batch_size, time_steps, features), stateful=True))
```

Listing 32.17: Set the stateful parameter on the LSTM layer

This same batch size must then be used later when evaluating the model and making predictions. For example:

```
model.predict(trainX, batch_size=batch_size)
```

Listing 32.18: Set batch size when making predictions

You can adapt the previous time step example to use a stateful LSTM. The full code listing is provided below.

```
# LSTM for international airline passengers problem with memory
import numpy as np
import matplotlib.pyplot as plt
from pandas import read_csv
import tensorflow as tf
from keras.models import Sequential
from keras.layers import Dense
from keras.layers import LSTM
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error
# convert an array of values into a dataset matrix
def create_dataset(dataset, look_back=1):
    dataX, dataY = [], []
    for i in range(len(dataset)-look_back-1):
        a = dataset[i:(i+look_back), 0]
        dataX.append(a)
        dataY.append(dataset[i + look_back, 0])
    return np.array(dataX), np.array(dataY)
# fix random seed for reproducibility
tf.random.set_seed(7)
# load the dataset
dataframe = read_csv('airline-passengers.csv', usecols=[1], engine='python')
dataset = dataframe.values
dataset = dataset.astype('float32')
# normalize the dataset
scaler = MinMaxScaler(feature_range=(0, 1))
dataset = scaler.fit_transform(dataset)
# split into train and test sets
train_size = int(len(dataset) * 0.67)
test_size = len(dataset) - train_size
train, test = dataset[0:train_size,:], dataset[train_size:len(dataset),:]
# reshape into X=t and Y=t+1
look_back = 3
trainX, trainY = create_dataset(train, look_back)
testX, testY = create_dataset(test, look_back)
# reshape input to be [samples, time steps, features]
trainX = np.reshape(trainX, (trainX.shape[0], trainX.shape[1], 1))
testX = np.reshape(testX, (testX.shape[0], testX.shape[1], 1))
# create and fit the LSTM network
batch_size = 1
model = Sequential()
model.add(LSTM(4, batch_input_shape=(batch_size, look_back, 1), stateful=True))
model.add(Dense(1))
```

```
model.compile(loss='mean_squared_error', optimizer='adam')
for i in range(100):
    model.fit(trainX, trainY, epochs=1, batch_size=batch_size, verbose=2, shuffle=False)
    model.reset_states()
# make predictions
trainPredict = model.predict(trainX, batch_size=batch_size)
model.reset_states()
testPredict = model.predict(testX, batch_size=batch_size)
# invert predictions
trainPredict = scaler.inverse_transform(trainPredict)
trainY = scaler.inverse_transform([trainY])
testPredict = scaler.inverse_transform(testPredict)
testY = scaler.inverse_transform([testY])
# calculate root mean squared error
trainScore = np.sqrt(mean_squared_error(trainY[0], trainPredict[:,0]))
print('Train Score: %.2f RMSE' % (trainScore))
testScore = np.sqrt(mean_squared_error(testY[0], testPredict[:,0]))
print('Test Score: %.2f RMSE' % (testScore))
# shift train predictions for plotting
trainPredictPlot = np.empty_like(dataset)
trainPredictPlot[:, :] = np.nan
trainPredictPlot[look_back:len(trainPredict)+look_back, :] = trainPredict
# shift test predictions for plotting
testPredictPlot = np.empty_like(dataset)
testPredictPlot[:, :] = np.nan
testPredictPlot[len(trainPredict)+(look_back*2)+1:len(dataset)-1, :] = testPredict
# plot baseline and predictions
plt.plot(scaler.inverse_transform(dataset))
plt.plot(trainPredictPlot)
plt.plot(testPredictPlot)
plt.show()
```

Listing 32.19: LSTM for the manual management of state

Running the example provides the following output:

```
...
92/92 - 0s - loss: 0.0024 - 46ms/epoch - 502us/step
92/92 - 0s - loss: 0.0023 - 49ms/epoch - 538us/step
92/92 - 0s - loss: 0.0023 - 47ms/epoch - 514us/step
92/92 - 0s - loss: 0.0023 - 48ms/epoch - 526us/step
92/92 - 0s - loss: 0.0022 - 48ms/epoch - 517us/step
92/92 - 0s - loss: 0.0022 - 48ms/epoch - 521us/step
92/92 - 0s - loss: 0.0022 - 47ms/epoch - 512us/step
92/92 - 0s - loss: 0.0021 - 50ms/epoch - 540us/step
92/92 - 0s - loss: 0.0021 - 47ms/epoch - 512us/step
92/92 - 0s - loss: 0.0021 - 52ms/epoch - 565us/step
92/92 [==============================] - 0s 448us/step
44/44 [==============================] - 0s 383us/step
Train Score: 24.48 RMSE
Test Score: 49.55 RMSE
```

Output 32.6: Sample output from the stateful LSTM model

You do see that results are better than some, worse than others. The model may need more modules and may need to be trained for more epochs to internalize the structure of the problem.

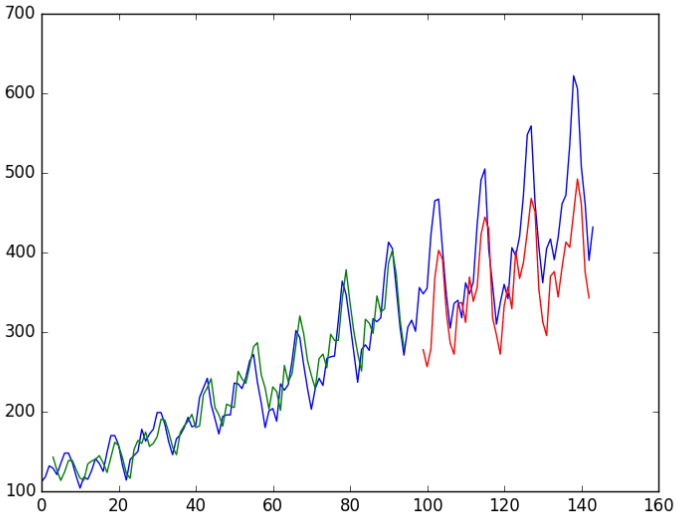

Figure 32.4: Stateful LSTM trained on regression formulation of passenger prediction problem

32.6 Stacked LSTMs with Memory Between Batches

Finally, let's take a look at one of the big benefits of LSTMs, the fact that they can be successfully trained when stacked into deep network architectures. LSTM networks can be stacked in Keras in the same way that other layer types can be stacked. One addition to the configuration that is required is that an LSTM layer prior to each subsequent LSTM layer must return the sequence. This can be done by setting the `return_sequences` parameter on the layer to `True`. You can extend the stateful LSTM in the previous section to have two layers, as follows:

```
model.add(LSTM(4, batch_input_shape=(batch_size, look_back, 1), stateful=True,
               return_sequences=True))
model.add(LSTM(4, batch_input_shape=(batch_size, look_back, 1), stateful=True))
```

Listing 32.20: Define a stacked LSTM model

The entire code listing is provided below for completeness.

```
# Stacked LSTM for international airline passengers problem with memory
import numpy as np
import matplotlib.pyplot as plt
from pandas import read_csv
import tensorflow as tf
from keras.models import Sequential
```

```
from keras.layers import Dense
from keras.layers import LSTM
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error
# convert an array of values into a dataset matrix
def create_dataset(dataset, look_back=1):
    dataX, dataY = [], []
    for i in range(len(dataset)-look_back-1):
        a = dataset[i:(i+look_back), 0]
        dataX.append(a)
        dataY.append(dataset[i + look_back, 0])
    return np.array(dataX), np.array(dataY)
# fix random seed for reproducibility
tf.random.set_seed(7)
# load the dataset
dataframe = read_csv('airline-passengers.csv', usecols=[1], engine='python')
dataset = dataframe.values
dataset = dataset.astype('float32')
# normalize the dataset
scaler = MinMaxScaler(feature_range=(0, 1))
dataset = scaler.fit_transform(dataset)
# split into train and test sets
train_size = int(len(dataset) * 0.67)
test_size = len(dataset) - train_size
train, test = dataset[0:train_size,:], dataset[train_size:len(dataset),:]
# reshape into X=t and Y=t+1
look_back = 3
trainX, trainY = create_dataset(train, look_back)
testX, testY = create_dataset(test, look_back)
# reshape input to be [samples, time steps, features]
trainX = np.reshape(trainX, (trainX.shape[0], trainX.shape[1], 1))
testX = np.reshape(testX, (testX.shape[0], testX.shape[1], 1))
# create and fit the LSTM network
batch_size = 1
model = Sequential()
model.add(LSTM(4, batch_input_shape=(batch_size, look_back, 1), stateful=True,
               return_sequences=True))
model.add(LSTM(4, batch_input_shape=(batch_size, look_back, 1), stateful=True))
model.add(Dense(1))
model.compile(loss='mean_squared_error', optimizer='adam')
for i in range(100):
    model.fit(trainX, trainY, epochs=1, batch_size=batch_size, verbose=2, shuffle=False)
    model.reset_states()
# make predictions
trainPredict = model.predict(trainX, batch_size=batch_size)
model.reset_states()
testPredict = model.predict(testX, batch_size=batch_size)
# invert predictions
trainPredict = scaler.inverse_transform(trainPredict)
trainY = scaler.inverse_transform([trainY])
testPredict = scaler.inverse_transform(testPredict)
testY = scaler.inverse_transform([testY])
# calculate root mean squared error
trainScore = np.sqrt(mean_squared_error(trainY[0], trainPredict[:,0]))
```

```
print('Train Score: %.2f RMSE' % (trainScore))
testScore = np.sqrt(mean_squared_error(testY[0], testPredict[:,0]))
print('Test Score: %.2f RMSE' % (testScore))
# shift train predictions for plotting
trainPredictPlot = np.empty_like(dataset)
trainPredictPlot[:, :] = np.nan
trainPredictPlot[look_back:len(trainPredict)+look_back, :] = trainPredict
# shift test predictions for plotting
testPredictPlot = np.empty_like(dataset)
testPredictPlot[:, :] = np.nan
testPredictPlot[len(trainPredict)+(look_back*2)+1:len(dataset)-1, :] = testPredict
# plot baseline and predictions
plt.plot(scaler.inverse_transform(dataset))
plt.plot(trainPredictPlot)
plt.plot(testPredictPlot)
plt.show()
```

Listing 32.21: Stacked LSTM model

Running the example produces the following output.

```
...
92/92 - 0s - loss: 0.0016 - 78ms/epoch - 849us/step
92/92 - 0s - loss: 0.0015 - 80ms/epoch - 874us/step
92/92 - 0s - loss: 0.0015 - 78ms/epoch - 843us/step
92/92 - 0s - loss: 0.0015 - 78ms/epoch - 845us/step
92/92 - 0s - loss: 0.0015 - 79ms/epoch - 859us/step
92/92 - 0s - loss: 0.0015 - 78ms/epoch - 848us/step
92/92 - 0s - loss: 0.0015 - 78ms/epoch - 844us/step
92/92 - 0s - loss: 0.0015 - 78ms/epoch - 852us/step
92/92 [==============================] - 0s 563us/step
44/44 [==============================] - 0s 453us/step
Train Score: 20.58 RMSE
Test Score: 55.99 RMSE
```

Output 32.7: Sample output from the stacked LSTM model

The predictions on the test dataset are again worse. This is more evidence to suggest the need for additional training epochs.

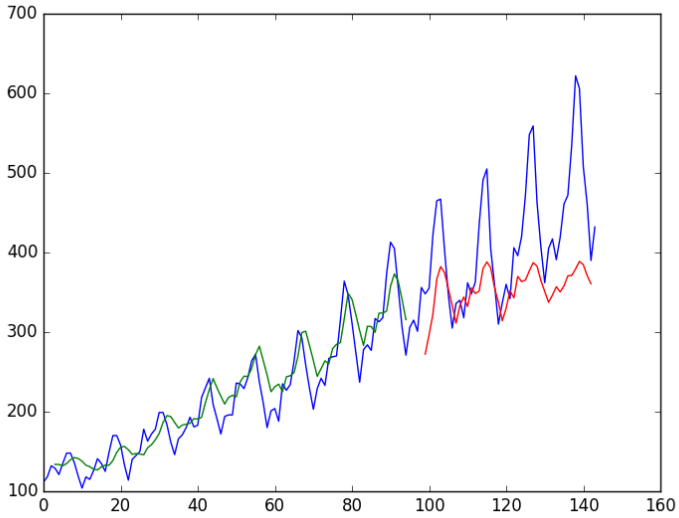

Figure 32.5: Stacked stateful LSTMs trained on regression formulation of passenger prediction problem

32.7 Further Reading

This section provides more resources on the topic if you are looking to go deeper.

Articles

LSTM layer. Keras API reference.
`https://keras.io/api/layers/recurrent_layers/lstm/`
Time series forecasting. TensorFlow tutorials.
`https://www.tensorflow.org/tutorials/structured_data/time_series`

32.8 Summary

In this chapter, you discovered how to develop LSTM recurrent neural networks for time series prediction in Python with the Keras deep learning network. Specifically, you learned:

- ▷ About the international airline passenger time series prediction problem
- ▷ How to create an LSTM for a regression and a window formulation of the time series problem
- ▷ How to create an LSTM with a time step formulation of the time series problem
- ▷ How to create an LSTM with state and stacked LSTMs with state to learn long sequences

In the next chapter, you will learn that LSTM is not only for time series problems, but also can help understanding a paragraph.

Project: Sequence Classification of Movie Reviews

33

Sequence classification is a predictive modeling problem where you have some sequence of inputs over space or time, and the task is to predict a category for the sequence. This problem is difficult because the sequences can vary in length, comprise a very large vocabulary of input symbols, and may require the model to learn the long term context or dependencies between symbols in the input sequence. In this chapter, you will discover how you can develop LSTM recurrent neural network models for sequence classification problems in Python using the Keras deep learning library. After completing this chapter, you will know:

- How to develop an LSTM model for a sequence classification problem
- How to reduce overfitting in your LSTM models through the use of dropout
- How to combine LSTM models with Convolutional Neural Networks that excel at learning spatial relationships

Let's get started.

Overview

This chapter is divided into five sections; they are:

- Problem Description
- Simple LSTM for Sequence Classification
- LSTM for Sequence Classification with Dropout
- Bidirectional LSTM for Sequence Classification
- LSTM and CNN for Sequence Classification

33.1 Problem Description

The problem that you will use to demonstrate sequence learning in this chapter is the IMDB movie review sentiment classification problem, introduced in Section 29.1.

Keras provides built-in access to the IMDB dataset. The `imdb.load_data()` function allows you to load the dataset in a format ready for use in neural networks and deep learning

models. The words have been replaced by integers that indicate the ordered frequency of each word in the dataset. The sentences in each review are therefore comprised of a sequence of integers.

Word Embedding

You will map each movie review into a real vector domain, a popular technique when working with text — called word embedding. This is a technique where words are encoded as real-valued vectors in a high dimensional space, where the similarity between words in terms of meaning translates to closeness in the vector space.

Keras provides a convenient way to convert positive integer representations of words into a word embedding by an `Embedding` layer. You will map each word onto a 32-length real valued vector. You will also limit the total number of words that you are interested in modeling to the 5000 most frequent words and zero out the rest. Finally, the sequence length (number of words) in each review varies, so you will constrain each review to be 500 words, truncating long reviews and padding the shorter reviews with zero values.

Now that you have defined your problem and how the data will be prepared and modeled, you are ready to develop an LSTM model to classify the sentiment of movie reviews.

33.2 Simple LSTM for Sequence Classification

You can quickly develop a small LSTM for the IMDB problem and achieve good accuracy. Let's start by importing the classes and functions required for this model and initializing the random number generator to a constant value to ensure you can easily reproduce the results.

```
import tensorflow as tf
from tensorflow.keras.datasets import imdb
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.layers import LSTM
from tensorflow.keras.layers import Embedding
from tensorflow.keras.preprocessing import sequence
# fix random seed for reproducibility
tf.random.set_seed(7)
```

Listing 33.1: Import classes and functions

You need to load the IMDB dataset. You are constraining the dataset to the top 5,000 words. You will also split the dataset into train (50%) and test (50%) sets.

```
...
# load the dataset but only keep the top n words, zero the rest
top_words = 5000
(X_train, y_train), (X_test, y_test) = imdb.load_data(num_words=top_words)
```

Listing 33.2: Load and split the dataset

Next, you need to truncate and pad the input sequences, so they are all the same length for modeling. The model will learn that the zero values carry no information. The sequences are

not the same length in terms of content, but same-length vectors are required to perform the computation in Keras.

```
...
# truncate and pad input sequences
max_review_length = 500
X_train = sequence.pad_sequences(X_train, maxlen=max_review_length)
X_test = sequence.pad_sequences(X_test, maxlen=max_review_length)
```

Listing 33.3: Left-pad sequences to all be the same length

You can now define, compile and fit your LSTM model. The first layer is the Embedded layer that uses 32-length vectors to represent each word. The next layer is the LSTM layer with 100 memory units (smart neurons). Finally, because this is a classification problem, you will use a `Dense` output layer with a single neuron and a sigmoid activation function to make 0 or 1 predictions for the two classes (good and bad) in the problem. Because it is a binary classification problem, log loss is used as the loss function (`binary_crossentropy` in Keras). The efficient ADAM optimization algorithm is used. The model is fit for only three epochs because it quickly overfits the problem. A large batch size of 64 reviews is used to space out weight updates.

```
...
# create the model
embedding_vecor_length = 32
model = Sequential()
model.add(Embedding(top_words, embedding_vecor_length, input_length=max_review_length))
model.add(LSTM(100))
model.add(Dense(1, activation='sigmoid'))
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
model.summary()
model.fit(X_train, y_train, validation_data=(X_test, y_test), epochs=3, batch_size=64)
```

Listing 33.4: Define and fit LSTM model for the IMDB dataset

Once fit, you can estimate the performance of the model on unseen reviews.

```
...
# Final evaluation of the model
scores = model.evaluate(X_test, y_test, verbose=0)
print("Accuracy: %.2f%%" % (scores[1]*100))
```

Listing 33.5: Evaluate the fit model

For completeness, here is the full code listing for this LSTM network on the IMDB dataset.

```
# LSTM for sequence classification in the IMDB dataset
import tensorflow as tf
from tensorflow.keras.datasets import imdb
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.layers import LSTM
from tensorflow.keras.layers import Embedding
```

```
from tensorflow.keras.preprocessing import sequence
# fix random seed for reproducibility
tf.random.set_seed(7)
# load the dataset but only keep the top n words, zero the rest
top_words = 5000
(X_train, y_train), (X_test, y_test) = imdb.load_data(num_words=top_words)
# truncate and pad input sequences
max_review_length = 500
X_train = sequence.pad_sequences(X_train, maxlen=max_review_length)
X_test = sequence.pad_sequences(X_test, maxlen=max_review_length)
# create the model
embedding_vecor_length = 32
model = Sequential()
model.add(Embedding(top_words, embedding_vecor_length, input_length=max_review_length))
model.add(LSTM(100))
model.add(Dense(1, activation='sigmoid'))
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
model.summary()
model.fit(X_train, y_train, epochs=3, batch_size=64)
# Final evaluation of the model
scores = model.evaluate(X_test, y_test, verbose=0)
print("Accuracy: %.2f%%" % (scores[1]*100))
```

Listing 33.6: Simple LSTM model for the IMDB dataset

Note: Your results may vary given the stochastic nature of the algorithm or evaluation procedure, or differences in numerical precision. Consider running the example a few times and compare the average outcome.

Running this example produces the following output.

```
Epoch 1/3
391/391 [==============================] - 124s 316ms/step - loss: 0.4525 - accuracy: 0.7794
Epoch 2/3
391/391 [==============================] - 124s 318ms/step - loss: 0.3117 - accuracy: 0.8706
Epoch 3/3
391/391 [==============================] - 126s 323ms/step - loss: 0.2526 - accuracy: 0.9003
Accuracy: 86.83%
```

Output 33.1: Output from running the simple LSTM model

You can see that this simple LSTM with little tuning achieves near state-of-the-art results on the IMDB problem. Importantly, this is a template that you can use to apply LSTM networks to your own sequence classification problems. Now, let's look at some extensions of this simple model that you may also want to bring to your own problems.

33.3 LSTM for Sequence Classification with Dropout

Recurrent neural networks like LSTM generally have the problem of overfitting. Dropout can be applied between layers using the `Dropout` Keras layer. You can do this easily by adding new

`Dropout` layers between the `Embedding` and `LSTM` layers and the `LSTM` and `Dense` output layers. For example:

```
model = Sequential()
model.add(Embedding(top_words, embedding_vecor_length, input_length=max_review_length))
model.add(Dropout(0.2))
model.add(LSTM(100))
model.add(Dropout(0.2))
model.add(Dense(1, activation='sigmoid'))
```

Listing 33.7: Define LSTM model with dropout layers

The full code listing example above with the addition of Dropout layers is as follows:

```
# LSTM with Dropout for sequence classification in the IMDB dataset
import tensorflow as tf
from tensorflow.keras.datasets import imdb
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.layers import LSTM
from tensorflow.keras.layers import Dropout
from tensorflow.keras.layers import Embedding
from tensorflow.keras.preprocessing import sequence
# fix random seed for reproducibility
tf.random.set_seed(7)
# load the dataset but only keep the top n words, zero the rest
top_words = 5000
(X_train, y_train), (X_test, y_test) = imdb.load_data(num_words=top_words)
# truncate and pad input sequences
max_review_length = 500
X_train = sequence.pad_sequences(X_train, maxlen=max_review_length)
X_test = sequence.pad_sequences(X_test, maxlen=max_review_length)
# create the model
embedding_vecor_length = 32
model = Sequential()
model.add(Embedding(top_words, embedding_vecor_length, input_length=max_review_length))
model.add(Dropout(0.2))
model.add(LSTM(100))
model.add(Dropout(0.2))
model.add(Dense(1, activation='sigmoid'))
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
model.summary()
model.fit(X_train, y_train, epochs=3, batch_size=64)
# Final evaluation of the model
scores = model.evaluate(X_test, y_test, verbose=0)
print("Accuracy: %.2f%%" % (scores[1]*100))
```

Listing 33.8: LSTM model with dropout for the IMDB dataset

Running this example provides the following output.

```
Epoch 1/3
391/391 [==============================] - 117s 297ms/step - loss: 0.4721 - accuracy: 0.7664
Epoch 2/3
391/391 [==============================] - 125s 319ms/step - loss: 0.2840 - accuracy: 0.8864
Epoch 3/3
391/391 [==============================] - 135s 346ms/step - loss: 0.3022 - accuracy: 0.8772
Accuracy: 85.66%
```

Output 33.2: Output from running the LSTM model with dropout layer

You can see dropout having the desired impact on training with a slightly slower trend in convergence and, in this case, a lower final accuracy. The model could probably use a few more epochs of training and may achieve a higher skill (try it and see). Alternately, dropout can be applied to the input and recurrent connections of the memory units with the LSTM precisely and separately. Keras provides this capability with parameters on the `LSTM` layer, the `dropout`, for configuring the input dropout and `recurrent_dropout` for configuring the recurrent dropout. For example, you can modify the first example to add dropout to the input and recurrent connections as follows:

```
model = Sequential()
model.add(Embedding(top_words, embedding_vecor_length, input_length=max_review_length))
model.add(LSTM(100, dropout=0.2, recurrent_dropout=0.2))
model.add(Dense(1, activation='sigmoid'))
```

Listing 33.9: Define LSTM model with dropout on gates

The full code listing with more precise LSTM dropout is listed below for completeness.

```
# LSTM with dropout for sequence classification in the IMDB dataset
import tensorflow as tf
from tensorflow.keras.datasets import imdb
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.layers import LSTM
from tensorflow.keras.layers import Embedding
from tensorflow.keras.preprocessing import sequence
# fix random seed for reproducibility
tf.random.set_seed(7)
# load the dataset but only keep the top n words, zero the rest
top_words = 5000
(X_train, y_train), (X_test, y_test) = imdb.load_data(num_words=top_words)
# truncate and pad input sequences
max_review_length = 500
X_train = sequence.pad_sequences(X_train, maxlen=max_review_length)
X_test = sequence.pad_sequences(X_test, maxlen=max_review_length)
# create the model
embedding_vecor_length = 32
model = Sequential()
model.add(Embedding(top_words, embedding_vecor_length, input_length=max_review_length))
model.add(LSTM(100, dropout=0.2, recurrent_dropout=0.2))
model.add(Dense(1, activation='sigmoid'))
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
model.summary()
```

```
model.fit(X_train, y_train, epochs=3, batch_size=64)
# Final evaluation of the model
scores = model.evaluate(X_test, y_test, verbose=0)
print("Accuracy: %.2f%%" % (scores[1]*100))
```

Listing 33.10: LSTM model with dropout on gates for the IMDB dataset

Running this example provides the following output.

```
Epoch 1/3
391/391 [==============================] - 220s 560ms/step - loss: 0.4605 - accuracy: 0.7784
Epoch 2/3
391/391 [==============================] - 219s 560ms/step - loss: 0.3158 - accuracy: 0.8773
Epoch 3/3
391/391 [==============================] - 219s 559ms/step - loss: 0.2734 - accuracy: 0.8930
Accuracy: 86.78%
```

Output 33.3: Output from running the LSTM model with dropout on the gates

You can see that the LSTM-specific dropout has a more pronounced effect on the convergence of the network than the layer-wise dropout. Like above, the number of epochs was kept constant and could be increased to see if the skill of the model could be further lifted. Dropout is a powerful technique for combating overfitting in your LSTM models, and it is a good idea to try both methods. Still, you may get better results with the gate-specific dropout provided in Keras.

33.4 Bidirectional LSTM for Sequence Classification

Sometimes, a sequence is better to be used in reversed order. In those case, you can simply reverse a vector `x` using the Python syntax `x[::-1]` before using is to train our LSTM network. Sometimes, neither the forward nor the reversed order works perfectly, but combining them will give better results. In this case, you will need a *bidirectional LSTM network*.

A bidirectional LSTM network is simply two separately LSTM networks; one feeds with forward sequence and another with reversed sequence. Then the output of the two LSTM networks is concatenated together before fed to the subsequent layers of the network. In Keras, we have the function `Bidirectional()` to clone a LSTM layer for forward-backward input and concatenate their output. For example,

```
model = Sequential()
model.add(Embedding(top_words, embedding_vecor_length, input_length=max_review_length))
model.add(Bidirectional(LSTM(100, dropout=0.2, recurrent_dropout=0.2)))
model.add(Dense(1, activation='sigmoid'))
```

Listing 33.11: Creating a bidirectional LSTM network

Since we created not one, but two LSTM with 100 units each, this network will take twice the amount of time to train. Depending on the problem, this additional cost may be justified.

The full code listing with adding the bidirectional LSTM to the last example is listed below for completeness.

```
# LSTM with dropout for sequence classification in the IMDB dataset
import tensorflow as tf
from tensorflow.keras.datasets import imdb
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.layers import LSTM
from tensorflow.keras.layers import Bidirectional
from tensorflow.keras.layers import Embedding
from tensorflow.keras.preprocessing import sequence
# fix random seed for reproducibility
tf.random.set_seed(7)
# load the dataset but only keep the top n words, zero the rest
top_words = 5000
(X_train, y_train), (X_test, y_test) = imdb.load_data(num_words=top_words)
# truncate and pad input sequences
max_review_length = 500
X_train = sequence.pad_sequences(X_train, maxlen=max_review_length)
X_test = sequence.pad_sequences(X_test, maxlen=max_review_length)
# create the model
embedding_vecor_length = 32
model = Sequential()
model.add(Embedding(top_words, embedding_vecor_length, input_length=max_review_length))
model.add(Bidirectional(LSTM(100, dropout=0.2, recurrent_dropout=0.2)))
model.add(Dense(1, activation='sigmoid'))
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
model.summary()
model.fit(X_train, y_train, epochs=3, batch_size=64)
# Final evaluation of the model
scores = model.evaluate(X_test, y_test, verbose=0)
print("Accuracy: %.2f%%" % (scores[1]*100))
```

Listing 33.12: Bidirectional LSTM model for the IMDB dataset

Running this example provides the following output.

```
Epoch 1/3
391/391 [==============================] - 405s 1s/step - loss: 0.4960 - accuracy: 0.7532
Epoch 2/3
391/391 [==============================] - 439s 1s/step - loss: 0.3075 - accuracy: 0.8744
Epoch 3/3
391/391 [==============================] - 430s 1s/step - loss: 0.2551 - accuracy: 0.9014
Accuracy: 87.69%
```

Output 33.4: Output from running the bidirectional LSTM model

It seems we can only get a slight improvement but with a significantly longer training time.

33.5 LSTM and CNN for Sequence Classification

Convolutional neural networks excel at learning the spatial structure in input data. The IMDB review data is a collection of sentences. It does have a one-dimensional spatial structure of a sequence of words in reviews, and the CNN may be able to pick out invariant features for the good and bad sentiment. This learned spatial feature may then be learned as sequences

by an LSTM layer. You can easily add a one-dimensional CNN and max pooling layers after the `Embedding` layer, which then feeds the consolidated features to the LSTM. You can use a smallish set of 32 features with a small filter length of 3. The pooling layer can use the standard length of 2 to halve the feature map size.

For example, you would create the model as follows:

```
model = Sequential()
model.add(Embedding(top_words, embedding_vecor_length, input_length=max_review_length))
model.add(Conv1D(32, kernel_size=3, padding='same', activation='relu'))
model.add(MaxPooling1D(pool_size=2))
model.add(LSTM(100))
model.add(Dense(1, activation='sigmoid'))
```

Listing 33.13: Define LSTM and CNN model

The full code listing with CNN and LSTM layers is listed below for completeness.

```
# LSTM and CNN for sequence classification in the IMDB dataset
import tensorflow as tf
from tensorflow.keras.datasets import imdb
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.layers import LSTM
from tensorflow.keras.layers import Conv1D
from tensorflow.keras.layers import MaxPooling1D
from tensorflow.keras.layers import Embedding
from tensorflow.keras.preprocessing import sequence
# fix random seed for reproducibility
tf.random.set_seed(7)
# load the dataset but only keep the top n words, zero the rest
top_words = 5000
(X_train, y_train), (X_test, y_test) = imdb.load_data(num_words=top_words)
# truncate and pad input sequences
max_review_length = 500
X_train = sequence.pad_sequences(X_train, maxlen=max_review_length)
X_test = sequence.pad_sequences(X_test, maxlen=max_review_length)
# create the model
embedding_vecor_length = 32
model = Sequential()
model.add(Embedding(top_words, embedding_vecor_length, input_length=max_review_length))
model.add(Conv1D(32, 3, padding='same', activation='relu'))
model.add(MaxPooling1D(pool_size=2))
model.add(LSTM(100))
model.add(Dense(1, activation='sigmoid'))
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
model.summary()
model.fit(X_train, y_train, epochs=3, batch_size=64)
# Final evaluation of the model
scores = model.evaluate(X_test, y_test, verbose=0)
print("Accuracy: %.2f%%" % (scores[1]*100))
```

Listing 33.14: LSTM and CNN model for the IMDB dataset

Running this example provides the following output.

```
Epoch 1/3
391/391 [==============================] - 65s 163ms/step - loss: 0.4213 - accuracy: 0.7950
Epoch 2/3
391/391 [==============================] - 66s 168ms/step - loss: 0.2490 - accuracy: 0.9026
Epoch 3/3
391/391 [==============================] - 73s 188ms/step - loss: 0.1979 - accuracy: 0.9261
Accuracy: 88.45%
```

Output 33.5: Output from running the LSTM and CNN model

You can see that you achieve slightly better results than the first example, although with fewer weights and faster training time. You might expect that even better results could be achieved if this example was further extended to use dropout.

33.6 Further Reading

Below are some resources if you are interested in diving deeper into sequence prediction or this specific example.

APIs

IMDB movie review sentiment classification dataset. Keras API reference.
https://keras.io/api/datasets/imdb/

LSTM layer. Keras API reference.
https://keras.io/api/layers/recurrent_layers/lstm/

Embedding Layer. Keras API reference.
https://keras.io/api/layers/core_layers/embedding/

Articles

Large Movie Review Dataset.
http://ai.stanford.edu/~amaas/data/sentiment/

François Chollet. *Bidirectional LSTM on IMDB*. Keras code examples, 2020.
https://keras.io/examples/nlp/bidirectional_lstm_imdb/

Books

Alex Graves. *Supervised Sequence Labelling with Recurrent Neural Networks*. Springer, 2012.
https://www.amazon.com/dp/3642247962/
(Preprint available online: https://www.cs.toronto.edu/~graves/preprint.pdf).

33.7 Summary

In this chapter, you discovered how to develop LSTM network models for sequence classification predictive modeling problems. Specifically, you learned:

- ▷ How to develop a simple single-layer LSTM model for the IMDB movie review sentiment classification problem

- ▷ How to extend your LSTM model with layer-wise and LSTM-specific dropout to reduce overfitting
- ▷ How to combine the spatial structure learning properties of a Convolutional Neural Network with the sequence learning of an LSTM

In the next chapter, you will look deeper into the states held by the LSTM network.

Understanding Stateful LSTM Recurrent Neural Networks

34

A powerful and popular recurrent neural network is the long short-term model network or LSTM. It is widely used because the architecture overcomes the vanishing and exploding gradient problem that plagues all recurrent neural networks, allowing very large and very deep networks to be created. Like other recurrent neural networks, LSTM networks maintain state, and the specifics of how this is implemented in a Keras framework can be confusing. In this chapter, you will discover exactly how state is maintained in LSTM networks by the Keras deep learning library. After reading this chapter, you will know:

- ▷ How to develop a naive LSTM network for a sequence prediction problem
- ▷ How to carefully manage state through batches and features with an LSTM network
- ▷ How to manually manage state in an LSTM network for stateful prediction

Let's get started.

Overview

This chapter is divided into seven sections; they are:

- ▷ Problem Description: Learn the Alphabet
- ▷ Naive LSTM for Learning One-Char to One-Char Mapping
- ▷ Naive LSTM for a Three-Char Feature Window to One-Char Mapping
- ▷ Naive LSTM for a Three-Char Time Step Window to One-Char Mapping
- ▷ LSTM State within a Batch
- ▷ Stateful LSTM for a One-Char to One-Char Mapping
- ▷ LSTM with Variable-Length Input to One-Char Output

34.1 Problem Description: Learn the Alphabet

In this chapter, you will develop and contrast a number of different LSTM recurrent neural network models. The context of these comparisons will be a simple sequence prediction problem of learning the alphabet. That is, given a letter of the alphabet, it will predict the next letter

of the alphabet. This is a simple sequence prediction problem that, once understood, can be generalized to other sequence prediction problems like time series prediction and sequence classification. Let's prepare the problem with some Python code you can reuse from example to example.

First, let's import all of the classes and functions you will use in this chapter.

```
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.layers import LSTM
from tensorflow.keras.utils import to_categorical
```

Listing 34.1: Import classes and functions

Next, you can seed the random number generator to ensure that the results are the same each time the code is executed.

```
# fix random seed for reproducibility
tf.random.set_seed(7)
```

Listing 34.2: Fix the random seed for reproducibility

You can now define your dataset, the alphabet. You define the alphabet in uppercase characters for readability. Neural networks model numbers, so you need to map the letters of the alphabet to integer values. You can do this easily by creating a dictionary (map) of the letter index to the character. You can also create a reverse lookup for converting predictions back into characters to be used later.

```
# define the raw dataset
alphabet = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
# create mapping of characters to integers (0-25) and the reverse
char_to_int = dict((c, i) for i, c in enumerate(alphabet))
int_to_char = dict((i, c) for i, c in enumerate(alphabet))
```

Listing 34.3: Define the alphabet dataset

Now, you need to create your input and output pairs on which to train your neural network. You can do this by defining an input sequence length, then reading sequences from the input alphabet sequence. For example, use an input length of 1. Starting at the beginning of the raw input data, you can read off the first letter "`A`" and the next letter as the prediction "`B`." You move along one character and repeat until we reach a prediction of "`Z`."

```
# prepare the dataset of input to output pairs encoded as integers
seq_length = 1
dataX = []
dataY = []
for i in range(0, len(alphabet) - seq_length, 1):
    seq_in = alphabet[i:i + seq_length]
    seq_out = alphabet[i + seq_length]
    dataX.append([char_to_int[char] for char in seq_in])
```

```
    dataY.append(char_to_int[seq_out])
    print(seq_in, '->', seq_out)
```

Listing 34.4: Create patterns from dataset

Also, print out the input pairs for sanity checking. Running the code to this point will produce the following output, summarizing input sequences of length 1 and a single output character.

```
A -> B
B -> C
C -> D
D -> E
E -> F
F -> G
G -> H
H -> I
I -> J
J -> K
K -> L
L -> M
M -> N
N -> O
O -> P
P -> Q
Q -> R
R -> S
S -> T
T -> U
U -> V
V -> W
W -> X
X -> Y
Y -> Z
```

Output 34.1: Sample alphabet training patterns

You need to reshape the NumPy array into a format expected by the LSTM networks, specifically [*samples, time steps, features*].

```
# reshape X to be [samples, time steps, features]
X = np.reshape(dataX, (len(dataX), seq_length, 1))
```

Listing 34.5: Reshape training patterns for LSTM layers

Once reshaped, you can then normalize the input integers to the range 0-to-1, the range of the sigmoid activation functions used by the LSTM network.

```
# normalize
X = X / float(len(alphabet))
```

Listing 34.6: Normalize training patterns

Finally, you can think of this problem as a sequence classification task, where each of the 26 letters represents a different class. As such, you can convert the output (y) to a one-hot encoding using the Keras built-in function `to_categorical()`.

```
# one-hot encode the output variable
y = to_categorical(dataY)
```

Listing 34.7: One-hot encode output patterns

You are now ready to fit different LSTM models.

34.2 Naive LSTM for Learning One-Char to One-Char Mapping

Let's start by designing a simple LSTM to learn how to predict the next character in the alphabet, given the context of just one character. You will frame the problem as a random collection of one-letter input to one-letter output pairs. As you will see, this is a problematic framing of the problem for the LSTM to learn. Let's define an LSTM network with 32 units and an output layer using the softmax activation function for making predictions. Because this is a multiclass classification problem, you can use the log loss function (called "`categorical_crossentropy`" in Keras) and optimize the network using the ADAM optimization function. The model is fit over 500 epochs with a batch size of 1.

```
# create and fit the model
model = Sequential()
model.add(LSTM(32, input_shape=(X.shape[1], X.shape[2])))
model.add(Dense(y.shape[1], activation='softmax'))
model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
model.fit(X, y, epochs=500, batch_size=1, verbose=2)
```

Listing 34.8: Define and fit LSTM network model

After you fit the model, you can evaluate and summarize the performance of the entire training dataset.

```
# summarize performance of the model
scores = model.evaluate(X, y, verbose=0)
print("Model Accuracy: %.2f%%" % (scores[1]*100))
```

Listing 34.9: Evaluate the fit LSTM network model

You can then re-run the training data through the network and generate predictions, converting both the input and output pairs back into their original character format to get a visual idea of how well the network learned the problem.

```
# demonstrate some model predictions
for pattern in dataX:
    x = np.reshape(pattern, (1, len(pattern), 1))
    x = x / float(len(alphabet))
    prediction = model.predict(x, verbose=0)
    index = np.argmax(prediction)
```

```
    result = int_to_char[index]
    seq_in = [int_to_char[value] for value in pattern]
    print(seq_in, "->", result)
```

Listing 34.10: Make predictions using the fit LSTM network

The entire code listing is provided below for completeness.

```
# Naive LSTM to learn one-char to one-char mapping
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.layers import LSTM
from tensorflow.keras.utils import to_categorical
# fix random seed for reproducibility
tf.random.set_seed(7)
# define the raw dataset
alphabet = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
# create mapping of characters to integers (0-25) and the reverse
char_to_int = dict((c, i) for i, c in enumerate(alphabet))
int_to_char = dict((i, c) for i, c in enumerate(alphabet))
# prepare the dataset of input to output pairs encoded as integers
seq_length = 1
dataX = []
dataY = []
for i in range(0, len(alphabet) - seq_length, 1):
    seq_in = alphabet[i:i + seq_length]
    seq_out = alphabet[i + seq_length]
    dataX.append([char_to_int[char] for char in seq_in])
    dataY.append(char_to_int[seq_out])
    print(seq_in, '->', seq_out)
# reshape X to be [samples, time steps, features]
X = np.reshape(dataX, (len(dataX), seq_length, 1))
# normalize
X = X / float(len(alphabet))
# one-hot encode the output variable
y = to_categorical(dataY)
# create and fit the model
model = Sequential()
model.add(LSTM(32, input_shape=(X.shape[1], X.shape[2])))
model.add(Dense(y.shape[1], activation='softmax'))
model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
model.fit(X, y, epochs=500, batch_size=1, verbose=2)
# summarize performance of the model
scores = model.evaluate(X, y, verbose=0)
print("Model Accuracy: %.2f%%" % (scores[1]*100))
# demonstrate some model predictions
for pattern in dataX:
    x = np.reshape(pattern, (1, len(pattern), 1))
    x = x / float(len(alphabet))
    prediction = model.predict(x, verbose=0)
    index = np.argmax(prediction)
    result = int_to_char[index]
```

```
    seq_in = [int_to_char[value] for value in pattern]
    print(seq_in, "->", result)
```

Listing 34.11: LSTM network for one-char to one-char mapping

Note: Your results may vary given the stochastic nature of the algorithm or evaluation procedure, or differences in numerical precision. Consider running the example a few times and compare the average outcome.

Running this example produces the following output.

```
Model Accuracy: 84.00%
['A'] -> B
['B'] -> C
['C'] -> D
['D'] -> E
['E'] -> F
['F'] -> G
['G'] -> H
['H'] -> I
['I'] -> J
['J'] -> K
['K'] -> L
['L'] -> M
['M'] -> N
['N'] -> O
['O'] -> P
['P'] -> Q
['Q'] -> R
['R'] -> S
['S'] -> T
['T'] -> U
['U'] -> W
['V'] -> Y
['W'] -> Z
['X'] -> Z
['Y'] -> Z
```

Output 34.2: Output from the one-char to one-char mapping

You can see that this problem is indeed difficult for the network to learn. The reason is that the poor LSTM units do not have any context to work with. Each input-output pattern is shown to the network in a random order, and the state of the network is reset after each pattern (each batch where each batch contains one pattern). This is an abuse of the LSTM network architecture, treating it like a standard multilayer perceptron. Next, let's try a different framing of the problem to provide more sequence to the network from which to learn.

34.3 Naive LSTM for a Three-Char Feature Window to One-Char Mapping

A popular approach to adding more context to data for multilayer perceptrons is to use the window method. This is where previous steps in the sequence are provided as additional input features to the network. You can try the same trick to provide more context to the LSTM network. Here, you will increase the sequence length from 1 to 3, for example:

```
...
# prepare the dataset of input to output pairs encoded as integers
seq_length = 3
```

Listing 34.12: Increase sequence length

This creates training patterns like this:

```
ABC -> D
BCD -> E
CDE -> F
```

Output 34.3: Sample of longer input sequence length

Each element in the sequence is then provided as a new input feature to the network. This requires a modification of how the input sequences are reshaped in the data preparation step:

```
...
# reshape X to be [samples, time steps, features]
X = np.reshape(dataX, (len(dataX), 1, seq_length))
```

Listing 34.13: Reshape input so sequence is features

It also requires modifying how the sample patterns are reshaped when demonstrating predictions from the model.

```
...
x = np.reshape(pattern, (1, 1, len(pattern)))
```

Listing 34.14: Reshape input for predictions so sequence is features

The entire code listing is provided below for completeness.

```
# Naive LSTM to learn three-char window to one-char mapping
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.layers import LSTM
from tensorflow.keras.utils import to_categorical
# fix random seed for reproducibility
tf.random.set_seed(7)
# define the raw dataset
alphabet = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
```

```
# create mapping of characters to integers (0-25) and the reverse
char_to_int = dict((c, i) for i, c in enumerate(alphabet))
int_to_char = dict((i, c) for i, c in enumerate(alphabet))
# prepare the dataset of input to output pairs encoded as integers
seq_length = 3
dataX = []
dataY = []
for i in range(0, len(alphabet) - seq_length, 1):
    seq_in = alphabet[i:i + seq_length]
    seq_out = alphabet[i + seq_length]
    dataX.append([char_to_int[char] for char in seq_in])
    dataY.append(char_to_int[seq_out])
    print(seq_in, '->', seq_out)
# reshape X to be [samples, time steps, features]
X = np.reshape(dataX, (len(dataX), 1, seq_length))
# normalize
X = X / float(len(alphabet))
# one-hot encode the output variable
y = to_categorical(dataY)
# create and fit the model
model = Sequential()
model.add(LSTM(32, input_shape=(X.shape[1], X.shape[2])))
model.add(Dense(y.shape[1], activation='softmax'))
model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
model.fit(X, y, epochs=500, batch_size=1, verbose=2)
# summarize performance of the model
scores = model.evaluate(X, y, verbose=0)
print("Model Accuracy: %.2f%%" % (scores[1]*100))
# demonstrate some model predictions
for pattern in dataX:
    x = np.reshape(pattern, (1, 1, len(pattern)))
    x = x / float(len(alphabet))
    prediction = model.predict(x, verbose=0)
    index = np.argmax(prediction)
    result = int_to_char[index]
    seq_in = [int_to_char[value] for value in pattern]
    print(seq_in, "->", result)
```

Listing 34.15: LSTM network for three-char features to one-char mapping

Running this example provides the following output.

```
Model Accuracy: 86.96%
['A', 'B', 'C'] -> D
['B', 'C', 'D'] -> E
['C', 'D', 'E'] -> F
['D', 'E', 'F'] -> G
['E', 'F', 'G'] -> H
['F', 'G', 'H'] -> I
['G', 'H', 'I'] -> J
['H', 'I', 'J'] -> K
['I', 'J', 'K'] -> L
['J', 'K', 'L'] -> M
['K', 'L', 'M'] -> N
```

```
['L', 'M', 'N'] -> O
['M', 'N', 'O'] -> P
['N', 'O', 'P'] -> Q
['O', 'P', 'Q'] -> R
['P', 'Q', 'R'] -> S
['Q', 'R', 'S'] -> T
['R', 'S', 'T'] -> U
['S', 'T', 'U'] -> V
['T', 'U', 'V'] -> Y
['U', 'V', 'W'] -> Z
['V', 'W', 'X'] -> Z
['W', 'X', 'Y'] -> Z
```

Output 34.4: Output from the three-char features to one-char mapping

You can see a slight lift in performance that may or may not be real. This is a simple problem that you were still not able to learn with LSTMs even with the window method. Again, this is a misuse of the LSTM network by a poor framing of the problem. Indeed, the sequences of letters are time steps of one feature rather than one time step of separate features. You have given more context to the network but not more sequence as expected. In the next section, you will give more context to the network in the form of time steps.

34.4 Naive LSTM for a Three-Char Time Step Window to One-Char Mapping

In Keras, the intended use of LSTMs is to provide context in the form of time steps, rather than windowed features like with other network types. You can take your first example and simply change the sequence length from 1 to 3.

```
seq_length = 3
```

Listing 34.16: Increase sequence length

Again, this creates input-output pairs that look like this:

```
ABC -> D
BCD -> E
CDE -> F
DEF -> G
```

Output 34.5: Sample of longer input sequence length

The difference is that the reshaping of the input data takes the sequence as a time step sequence of one feature rather than a single time step of multiple features.

```
# reshape X to be [samples, time steps, features]
X = np.reshape(dataX, (len(dataX), seq_length, 1))
```

Listing 34.17: Reshape input so sequence is time steps

This is the correct intended use of providing sequence context to your LSTM in Keras. The full code example is provided below for completeness.

```
# Naive LSTM to learn three-char time steps to one-char mapping
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.layers import LSTM
from tensorflow.keras.utils import to_categorical
# fix random seed for reproducibility
tf.random.set_seed(7)
# define the raw dataset
alphabet = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
# create mapping of characters to integers (0-25) and the reverse
char_to_int = dict((c, i) for i, c in enumerate(alphabet))
int_to_char = dict((i, c) for i, c in enumerate(alphabet))
# prepare the dataset of input to output pairs encoded as integers
seq_length = 3
dataX = []
dataY = []
for i in range(0, len(alphabet) - seq_length, 1):
    seq_in = alphabet[i:i + seq_length]
    seq_out = alphabet[i + seq_length]
    dataX.append([char_to_int[char] for char in seq_in])
    dataY.append(char_to_int[seq_out])
    print(seq_in, '->', seq_out)
# reshape X to be [samples, time steps, features]
X = np.reshape(dataX, (len(dataX), seq_length, 1))
# normalize
X = X / float(len(alphabet))
# one-hot encode the output variable
y = to_categorical(dataY)
# create and fit the model
model = Sequential()
model.add(LSTM(32, input_shape=(X.shape[1], X.shape[2])))
model.add(Dense(y.shape[1], activation='softmax'))
model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
model.fit(X, y, epochs=500, batch_size=1, verbose=2)
# summarize performance of the model
scores = model.evaluate(X, y, verbose=0)
print("Model Accuracy: %.2f%%" % (scores[1]*100))
# demonstrate some model predictions
for pattern in dataX:
    x = np.reshape(pattern, (1, len(pattern), 1))
    x = x / float(len(alphabet))
    prediction = model.predict(x, verbose=0)
    index = np.argmax(prediction)
    result = int_to_char[index]
    seq_in = [int_to_char[value] for value in pattern]
    print(seq_in, "->", result)
```

Listing 34.18: LSTM network for three-char time steps to one-char mapping

Running this example provides the following output.

```
Model Accuracy: 100.00%
['A', 'B', 'C'] -> D
['B', 'C', 'D'] -> E
['C', 'D', 'E'] -> F
['D', 'E', 'F'] -> G
['E', 'F', 'G'] -> H
['F', 'G', 'H'] -> I
['G', 'H', 'I'] -> J
['H', 'I', 'J'] -> K
['I', 'J', 'K'] -> L
['J', 'K', 'L'] -> M
['K', 'L', 'M'] -> N
['L', 'M', 'N'] -> O
['M', 'N', 'O'] -> P
['N', 'O', 'P'] -> Q
['O', 'P', 'Q'] -> R
['P', 'Q', 'R'] -> S
['Q', 'R', 'S'] -> T
['R', 'S', 'T'] -> U
['S', 'T', 'U'] -> V
['T', 'U', 'V'] -> W
['U', 'V', 'W'] -> X
['V', 'W', 'X'] -> Y
['W', 'X', 'Y'] -> Z
```

Output 34.6: Output from the three-char time steps to one-char mapping

You can see that the model learns the problem perfectly as evidenced by the model evaluation and the example predictions. But it has learned a simpler problem. Specifically, it has learned to predict the next letter from a sequence of three letters in the alphabet. It can be shown any random sequence of three letters from the alphabet and predict the next letter.

It cannot actually enumerate the alphabet. It's possible a large enough multilayer perceptron network might be able to learn the same mapping using the window method. The LSTM networks are stateful. They should be able to learn the whole alphabet sequence, but by default, the Keras implementation resets the network state after each training batch.

34.5 LSTM State within a Batch

The Keras implementation of LSTMs resets the state of the network after each batch. This suggests that if you had a batch size large enough to hold all input patterns and if all the input patterns were ordered sequentially, the LSTM could use the context of the sequence within the batch to better learn the sequence. You can demonstrate this easily by modifying the first example for learning a one-to-one mapping and increasing the batch size from 1 to the size of the training dataset. Additionally, Keras shuffles the training dataset before each training epoch. To ensure the training data patterns remain sequential, you can disable this shuffling.

```
...
model.fit(X, y, epochs=5000, batch_size=len(dataX), verbose=2, shuffle=False)
```

Listing 34.19: Increase base size to cover entire dataset

The network will learn the mapping of characters using within-batch sequence, but this context will not be available to the network when making predictions. You can evaluate both the ability of the network to make predictions randomly and in sequence. The full code example is provided below for completeness.

```
# Naive LSTM to learn one-char to one-char mapping with all data in each batch
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.layers import LSTM
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.preprocessing.sequence import pad_sequences
# fix random seed for reproducibility
tf.random.set_seed(7)
# define the raw dataset
alphabet = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
# create mapping of characters to integers (0-25) and the reverse
char_to_int = dict((c, i) for i, c in enumerate(alphabet))
int_to_char = dict((i, c) for i, c in enumerate(alphabet))
# prepare the dataset of input to output pairs encoded as integers
seq_length = 1
dataX = []
dataY = []
for i in range(0, len(alphabet) - seq_length, 1):
    seq_in = alphabet[i:i + seq_length]
    seq_out = alphabet[i + seq_length]
    dataX.append([char_to_int[char] for char in seq_in])
    dataY.append(char_to_int[seq_out])
    print(seq_in, '->', seq_out)
# convert list of lists to array and pad sequences if needed
X = pad_sequences(dataX, maxlen=seq_length, dtype='float32')
# reshape X to be [samples, time steps, features]
X = np.reshape(dataX, (X.shape[0], seq_length, 1))
# normalize
X = X / float(len(alphabet))
# one-hot encode the output variable
y = to_categorical(dataY)
# create and fit the model
model = Sequential()
model.add(LSTM(16, input_shape=(X.shape[1], X.shape[2])))
model.add(Dense(y.shape[1], activation='softmax'))
model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
model.fit(X, y, epochs=5000, batch_size=len(dataX), verbose=2, shuffle=False)
# summarize performance of the model
scores = model.evaluate(X, y, verbose=0)
print("Model Accuracy: %.2f%%" % (scores[1]*100))
# demonstrate some model predictions
```

```
for pattern in dataX:
    x = np.reshape(pattern, (1, len(pattern), 1))
    x = x / float(len(alphabet))
    prediction = model.predict(x, verbose=0)
    index = np.argmax(prediction)
    result = int_to_char[index]
    seq_in = [int_to_char[value] for value in pattern]
    print(seq_in, "->", result)
# demonstrate predicting random patterns
print("Test a Random Pattern:")
for i in range(0,20):
    pattern_index = np.random.randint(len(dataX))
    pattern = dataX[pattern_index]
    x = np.reshape(pattern, (1, len(pattern), 1))
    x = x / float(len(alphabet))
    prediction = model.predict(x, verbose=0)
    index = np.argmax(prediction)
    result = int_to_char[index]
    seq_in = [int_to_char[value] for value in pattern]
    print(seq_in, "->", result)
```

Listing 34.20: LSTM network for one-char to one-char mapping within batch

Running the example provides the following output.

```
Model Accuracy: 100.00%
['A'] -> B
['B'] -> C
['C'] -> D
['D'] -> E
['E'] -> F
['F'] -> G
['G'] -> H
['H'] -> I
['I'] -> J
['J'] -> K
['K'] -> L
['L'] -> M
['M'] -> N
['N'] -> O
['O'] -> P
['P'] -> Q
['Q'] -> R
['R'] -> S
['S'] -> T
['T'] -> U
['U'] -> V
['V'] -> W
['W'] -> X
['X'] -> Y
['Y'] -> Z
Test a Random Pattern:
['T'] -> U
['V'] -> W
```

```
['M'] -> N
['Q'] -> R
['D'] -> E
['V'] -> W
['T'] -> U
['U'] -> V
['J'] -> K
['F'] -> G
['N'] -> O
['B'] -> C
['M'] -> N
['F'] -> G
['F'] -> G
['P'] -> Q
['A'] -> B
['K'] -> L
['W'] -> X
['E'] -> F
```

Output 34.7: Output from the one-char to one-char mapping within batch

As expected, the network is able to use the within-sequence context to learn the alphabet, achieving 100% accuracy on the training data. Importantly, the network can make accurate predictions for the next letter in the alphabet for randomly selected characters. Very impressive.

34.6 Stateful LSTM for a One-Char to One-Char Mapping

You have seen that you can break up the raw data into fixed-size sequences and that this representation can be learned by the LSTM but only to learn random mappings of 3 characters to 1 character. You have also seen that you can pervert the batch size to offer more sequence to the network, but only during training. Ideally, you want to expose the network to the entire sequence and let it learn the inter-dependencies rather than you defining those dependencies explicitly in the framing of the problem.

You can do this in Keras by making the LSTM layers stateful and manually resetting the state of the network at the end of the epoch, which is also the end of the training sequence. This is truly how the LSTM networks are intended to be used. You first need to define your LSTM layer as stateful. In so doing, you must explicitly specify the batch size as a dimension on the input shape. This also means that when you evaluate the network or make predictions, you must also specify and adhere to this same batch size. This is not a problem now as you are using a batch size of 1. This could introduce difficulties when making predictions when the batch size is not one, as predictions will need to be made in the batch and the sequence.

```
...
batch_size = 1
model.add(LSTM(50, batch_input_shape=(batch_size, X.shape[1], X.shape[2]), stateful=True))
```

Listing 34.21: Define a stateful LSTM layer

An important difference in training the stateful LSTM is that you manually train it one epoch at a time and reset the state after each epoch. You can do this in a `for` loop. Again, do not shuffle the input, preserving the sequence in which the input training data was created.

```
for i in range(300):
    model.fit(X, y, epochs=1, batch_size=batch_size, verbose=2, shuffle=False)
    model.reset_states()
```

Listing 34.22: Manually manage LSTM state for each epoch

As mentioned, you specify the batch size when evaluating the performance of the network on the entire training dataset.

```
# summarize performance of the model
scores = model.evaluate(X, y, batch_size=batch_size, verbose=0)
model.reset_states()
print("Model Accuracy: %.2f%%" % (scores[1]*100))
```

Listing 34.23: Evaluate model using pre-defined batch size

Finally, you can demonstrate that the network has indeed learned the entire alphabet. You can seed it with the first letter "`A`," request a prediction, feed the prediction back in as an input, and repeat the process all the way to "`Z`."

```
# demonstrate some model predictions
seed = [char_to_int[alphabet[0]]]
for i in range(0, len(alphabet)-1):
    x = np.reshape(seed, (1, len(seed), 1))
    x = x / float(len(alphabet))
    prediction = model.predict(x, verbose=0)
    index = np.argmax(prediction)
    print(int_to_char[seed[0]], "->", int_to_char[index])
    seed = [index]
model.reset_states()
```

Listing 34.24: Seed network and make prediction from A to Z

You can also see if the network can make predictions starting from an arbitrary letter.

```
# demonstrate a random starting point
letter = "K"
seed = [char_to_int[letter]]
print("New start: ", letter)
for i in range(0, 5):
    x = np.reshape(seed, (1, len(seed), 1))
    x = x / float(len(alphabet))
    prediction = model.predict(x, verbose=0)
    index = np.argmax(prediction)
    print(int_to_char[seed[0]], "->", int_to_char[index])
    seed = [index]
model.reset_states()
```

Listing 34.25: Seed network with a random letter and a sequence of predictions

The entire code listing is provided below for completeness.

```
# Stateful LSTM to learn one-char to one-char mapping
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.layers import LSTM
from tensorflow.keras.utils import to_categorical
# fix random seed for reproducibility
tf.random.set_seed(7)
# define the raw dataset
alphabet = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
# create mapping of characters to integers (0-25) and the reverse
char_to_int = dict((c, i) for i, c in enumerate(alphabet))
int_to_char = dict((i, c) for i, c in enumerate(alphabet))
# prepare the dataset of input to output pairs encoded as integers
seq_length = 1
dataX = []
dataY = []
for i in range(0, len(alphabet) - seq_length, 1):
    seq_in = alphabet[i:i + seq_length]
    seq_out = alphabet[i + seq_length]
    dataX.append([char_to_int[char] for char in seq_in])
    dataY.append(char_to_int[seq_out])
    print(seq_in, '->', seq_out)
# reshape X to be [samples, time steps, features]
X = np.reshape(dataX, (len(dataX), seq_length, 1))
# normalize
X = X / float(len(alphabet))
# one-hot encode the output variable
y = to_categorical(dataY)
# create and fit the model
batch_size = 1
model = Sequential()
model.add(LSTM(50, batch_input_shape=(batch_size, X.shape[1], X.shape[2]), stateful=True))
model.add(Dense(y.shape[1], activation='softmax'))
model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
for i in range(300):
    model.fit(X, y, epochs=1, batch_size=batch_size, verbose=2, shuffle=False)
    model.reset_states()
# summarize performance of the model
scores = model.evaluate(X, y, batch_size=batch_size, verbose=0)
model.reset_states()
print("Model Accuracy: %.2f%%" % (scores[1]*100))
# demonstrate some model predictions
seed = [char_to_int[alphabet[0]]]
for i in range(0, len(alphabet)-1):
    x = np.reshape(seed, (1, len(seed), 1))
    x = x / float(len(alphabet))
    prediction = model.predict(x, verbose=0)
    index = np.argmax(prediction)
    print(int_to_char[seed[0]], "->", int_to_char[index])
```

```
    seed = [index]
model.reset_states()
# demonstrate a random starting point
letter = "K"
seed = [char_to_int[letter]]
print("New start: ", letter)
for i in range(0, 5):
    x = np.reshape(seed, (1, len(seed), 1))
    x = x / float(len(alphabet))
    prediction = model.predict(x, verbose=0)
    index = np.argmax(prediction)
    print(int_to_char[seed[0]], "->", int_to_char[index])
    seed = [index]
model.reset_states()
```

Listing 34.26: Stateful LSTM network for one-char to one-char mapping

Running the example provides the following output.

```
Model Accuracy: 100.00%
A -> B
B -> C
C -> D
D -> E
E -> F
F -> G
G -> H
H -> I
I -> J
J -> K
K -> L
L -> M
M -> N
N -> O
O -> P
P -> Q
Q -> R
R -> S
S -> T
T -> U
U -> V
V -> W
W -> X
X -> Y
Y -> Z
New start:  K
K -> B
B -> C
C -> D
D -> E
E -> F
```

Output 34.8: Output from the stateful LSTM for one-char to one-char mapping

You can see that the network has memorized the entire alphabet perfectly. It used the context of the samples themselves and learned whatever dependency it needed to predict the next character in the sequence. You can also see that if you seed the network with the first letter, it can correctly rattle off the rest of the alphabet. You can also see that it has only learned the full alphabet sequence and from a cold start. When asked to predict the next letter from "K," it predicts "B" and falls back into regurgitating the entire alphabet. To truly predict "K," the state of the network would need to be warmed up and iteratively fed the letters from "A" to "J." This reveals that you could achieve the same effect with a *stateless* LSTM by preparing training data like this:

```
---a -> b
--ab -> c
-abc -> d
abcd -> e
```

Output 34.9: Sample of equivalent training data for non-stateful LSTM layers

Here, the input sequence is fixed at 25 (a to y to predict z) and patterns are prefixed with zero-padding. Finally, this raises the question of training an LSTM network using variable length input sequences to predict the next character.

34.7 LSTM with Variable-Length Input to One-Char Output

In the previous section, you discovered that the Keras *stateful* LSTM was really only a shortcut to replaying the first n-sequences but didn't really help us learn a generic model of the alphabet. In this section, you will explore a variation of the *stateless* LSTM that learns random subsequences of the alphabet and an effort to build a model that can be given arbitrary letters or subsequences of letters and predict the next letter in the alphabet.

Firstly, you are changing the framing of the problem. To simplify, you will define a maximum input sequence length and set it to a small value like 5 to speed up training. This defines the maximum length of subsequences of the alphabet that will be drawn for training. In extensions, this could just be set to the full alphabet (26) or longer if you allow looping back to the start of the sequence. You also need to define the number of random sequences to create — in this case, 1000. This, too, could be more or less. It's likely fewer patterns are actually required.

```
...
# prepare the dataset of input to output pairs encoded as integers
num_inputs = 1000
max_len = 5
dataX = []
dataY = []
for i in range(num_inputs):
    start = np.random.randint(len(alphabet)-2)
    end = np.random.randint(start, min(start+max_len,len(alphabet)-1))
    sequence_in = alphabet[start:end+1]
    sequence_out = alphabet[end + 1]
    dataX.append([char_to_int[char] for char in sequence_in])
```

```
    dataY.append(char_to_int[sequence_out])
    print(sequence_in, '->', sequence_out)
```

Listing 34.27: Create dataset of variable length input sequences

Running this code in the broader context will create input patterns that look like the following:

```
PQRST -> U
W -> X
O -> P
OPQ -> R
IJKLM -> N
QRSTU -> V
ABCD -> E
X -> Y
GHIJ -> K
```

Output 34.10: Sample of variable length input sequences

The input sequences vary in length between 1 and `max_len` and therefore require zero padding. Here, use left-hand-side (prefix) padding with the Keras built-in `pad_sequences()` function.

```
...
X = pad_sequences(dataX, maxlen=max_len, dtype='float32')
```

Listing 34.28: Left-pad variable length input sequences

The trained model is evaluated on randomly selected input patterns. This could just as easily be new randomly generated sequences of characters. This could also be a linear sequence seeded with "A" with outputs fed back in as single character inputs. The full code listing is provided below for completeness.

```
# LSTM with Variable Length Input Sequences to One Character Output
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.layers import LSTM
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.preprocessing.sequence import pad_sequences
# fix random seed for reproducibility
np.random.seed(7)
tf.random.set_seed(7)
# define the raw dataset
alphabet = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
# create mapping of characters to integers (0-25) and the reverse
char_to_int = dict((c, i) for i, c in enumerate(alphabet))
int_to_char = dict((i, c) for i, c in enumerate(alphabet))
# prepare the dataset of input to output pairs encoded as integers
num_inputs = 1000
max_len = 5
dataX = []
dataY = []
```

```
for i in range(num_inputs):
    start = np.random.randint(len(alphabet)-2)
    end = np.random.randint(start, min(start+max_len,len(alphabet)-1))
    sequence_in = alphabet[start:end+1]
    sequence_out = alphabet[end + 1]
    dataX.append([char_to_int[char] for char in sequence_in])
    dataY.append(char_to_int[sequence_out])
    print(sequence_in, '->', sequence_out)
# convert list of lists to array and pad sequences if needed
X = pad_sequences(dataX, maxlen=max_len, dtype='float32')
# reshape X to be [samples, time steps, features]
X = np.reshape(X, (X.shape[0], max_len, 1))
# normalize
X = X / float(len(alphabet))
# one-hot encode the output variable
y = to_categorical(dataY)
# create and fit the model
batch_size = 1
model = Sequential()
model.add(LSTM(32, input_shape=(X.shape[1], 1)))
model.add(Dense(y.shape[1], activation='softmax'))
model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
model.fit(X, y, epochs=500, batch_size=batch_size, verbose=2)
# summarize performance of the model
scores = model.evaluate(X, y, verbose=0)
print("Model Accuracy: %.2f%%" % (scores[1]*100))
# demonstrate some model predictions
for i in range(20):
    pattern_index = np.random.randint(len(dataX))
    pattern = dataX[pattern_index]
    x = pad_sequences([pattern], maxlen=max_len, dtype='float32')
    x = np.reshape(x, (1, max_len, 1))
    x = x / float(len(alphabet))
    prediction = model.predict(x, verbose=0)
    index = np.argmax(prediction)
    result = int_to_char[index]
    seq_in = [int_to_char[value] for value in pattern]
    print(seq_in, "->", result)
```

Listing 34.29: LSTM network for variable length sequence to one-char mapping

Running this code produces the following output:

```
Model Accuracy: 98.90%
['Q', 'R'] -> S
['W', 'X'] -> Y
['W', 'X'] -> Y
['C', 'D'] -> E
['E'] -> F
['S', 'T', 'U'] -> V
['G', 'H', 'I', 'J', 'K'] -> L
['O', 'P', 'Q', 'R', 'S'] -> T
['C', 'D'] -> E
['O'] -> P
```

```
['N', 'O', 'P'] -> Q
['D', 'E', 'F', 'G', 'H'] -> I
['X'] -> Y
['K'] -> L
['M'] -> N
['R'] -> T
['K'] -> L
['E', 'F', 'G'] -> H
['Q'] -> R
['Q', 'R', 'S'] -> T
```

Output 34.11: Output for the LSTM network for variable length sequences to one-char mapping

You can see that although the model did not learn the alphabet perfectly from the randomly generated subsequences, it did very well. The model was not tuned and might require more training, a larger network, or both (an exercise for the reader). This is a good natural extension to the "*all sequential input examples in each batch*" alphabet model learned above in that it can handle ad hoc queries, but this time of arbitrary sequence length (up to the max length).

34.8 Further Reading

This section provides more resources on the topic if you are looking to go deeper.

APIs

LSTM layer. Keras API reference.
https://keras.io/api/layers/recurrent_layers/lstm/

34.9 Summary

In this chapter, you discovered LSTM recurrent neural networks in Keras and how they manage state. Specifically, you learned:

- ▷ How to develop a naive LSTM network for one-character to one-character prediction
- ▷ How to configure a naive LSTM to learn a sequence across time steps within a sample
- ▷ How to configure an LSTM to learn a sequence across samples by manually managing state

In the next chapter, you will put everything together to build a text generation program using LSTM.

Project: Text Generation with Alice in the Wonderland

35

Recurrent neural networks can also be used as generative models. This means that in addition to being used for predictive models (making predictions), they can learn the sequences of a problem and then generate entirely new plausible sequences for the problem domain. Generative models like this are useful not only to study how well a model has learned a problem but also to learn more about the problem domain itself.

In this chapter, you will discover how to create a generative model for text, character-by-character using LSTM recurrent neural networks in Python with Keras. After reading this chapter, you will know:

- ▷ Where to download a free corpus of text that you can use to train text generative models
- ▷ How to frame the problem of text sequences to a recurrent neural network generative model
- ▷ How to develop an LSTM to generate plausible text sequences for a given problem

Let's get started.

Note: You may want to speed up the computation for this chapter by using GPU rather than CPU hardware, such as the process described in Appendix C. This is a suggestion, not a requirement. The code will work just fine on the CPU.

Overview

This chapter is divided into five sections; they are:

- ▷ Problem Description: Project Gutenberg
- ▷ Develop a Small LSTM Recurrent Neural Network
- ▷ Generating Text with an LSTM Network
- ▷ Larger LSTM Recurrent Neural Network
- ▷ Extension Ideas to Improve the Model

35.1 Problem Description: Project Gutenberg

Many of the classical texts are no longer protected under copyright. This means you can download all the text for these books for free and use them in experiments, like creating generative models. Perhaps the best place to get access to free books that are no longer protected by copyright is Project Gutenberg. In this chapter, you will use a favorite book from childhood as the dataset: Alice's Adventures in Wonderland by Lewis Carroll[1].

You will learn the dependencies between characters and the conditional probabilities of characters in sequences so that you can, in turn, generate wholly new and original sequences of characters. This chapter is a lot of fun, and repeating these experiments with other books from Project Gutenberg is recommended. These experiments are not limited to text; you can also experiment with other ASCII data, such as computer source code, marked-up documents in LaTeX, HTML or Markdown, and more.

You can download the complete text in ASCII format (Plaintext UTF-8) for this book for free and place it in your working directory with the filename `wonderland.txt`. Now, you need to prepare the dataset ready for modeling. Project Gutenberg adds a standard header and footer to each book, which is not part of the original text. Open the file in a text editor and delete the header and footer. The header is obvious and ends with the text:

```
*** START OF THIS PROJECT GUTENBERG EBOOK ALICE'S ADVENTURES IN WONDERLAND ***
```

Output 35.1: Signal of the end of the file handler

The footer is all the text after the line of text that says:

```
THE END
```

Output 35.2: Signal of the start of the file footer

You should be left with a text file that has about 3,330 lines of text.

35.2 Develop a Small LSTM Recurrent Neural Network

In this section, you will develop a simple LSTM network to learn sequences of characters from *Alice in Wonderland*. In the next section, you will use this model to generate new sequences of characters. Let's start by importing the classes and functions you will use to train your model.

```
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.layers import Dropout
from tensorflow.keras.layers import LSTM
from tensorflow.keras.callbacks import ModelCheckpoint
```

[1] https://www.gutenberg.org/ebooks/11

```
from tensorflow.keras.utils import to_categorical
...
```

Listing 35.1: Import classes and functions

Next, you need to load the ASCII text for the book into memory and convert all of the characters to lowercase to reduce the vocabulary the network must learn.

```
...
# load ascii text and covert to lowercase
filename = "wonderland.txt"
raw_text = open(filename, 'r', encoding='utf-8').read()
raw_text = raw_text.lower()
```

Listing 35.2: Load the dataset and convert to lowercase

Now that the book is loaded, you must prepare the data for modeling by the neural network. You cannot model the characters directly; instead, you must convert the characters to integers. You can do this easily by first creating a set of all of the distinct characters in the book, then creating a map of each character to a unique integer.

```
...
# create mapping of unique chars to integers
chars = sorted(list(set(raw_text)))
char_to_int = dict((c, i) for i, c in enumerate(chars))
```

Listing 35.3: Create char-to-integer mapping

For example, the list of unique sorted lowercase characters in the book is as follows:

```
['\n', '\r', ' ', '!', '"', "'", '(', ')', '*', ',', '-', '.', ':', ';', '?', '[', ']',
 '_', 'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l', 'm', 'n', 'o', 'p',
 'q', 'r', 's', 't', 'u', 'v', 'w', 'x', 'y', 'z', '\xbb', '\xbf', '\xef']
```

Output 35.3: List of unique characters in the dataset

You can see that there may be some characters that we could remove to further clean up the dataset to reduce the vocabulary, which may improve the modeling process. Now that the book has been loaded and the mapping prepared, you can summarize the dataset.

```
...
n_chars = len(raw_text)
n_vocab = len(chars)
print("Total Characters: ", n_chars)
print("Total Vocab: ", n_vocab)
```

Listing 35.4: Summarize the loaded dataset

Running the code to this point produces the following output.

```
Total Characters:  147674
Total Vocab:  47
```

Output 35.4: Output from summarize the dataset

You can see the book has just under 150,000 characters, and when converted to lowercase, there are only 47 distinct characters in the vocabulary for the network to learn — much more than the 26 in the alphabet. You now need to define the training data for the network. There is a lot of flexibility in how you choose to break up the text and expose it to the network during training. In this chapter, you will split the book text up into subsequences with a fixed length of 100 characters, an arbitrary length. You could just as easily split the data by sentences, padding the shorter sequences and truncating the longer ones.

Each training pattern of the network comprises 100 time steps of one character (X) followed by one character output (y). When creating these sequences, we slide this window along the whole book one character at a time, allowing each character a chance to be learned from the 100 characters that preceded it (except the first 100 characters, of course). For example, if the sequence length is 5 (for simplicity), then the first two training patterns would be as follows:

```
CHAPT -> E
HAPTE -> R
```

Output 35.5: Example of sequence construction

As you split the book into these sequences, you convert the characters to integers using the lookup table you prepared earlier.

```
...
# prepare the dataset of input to output pairs encoded as integers
seq_length = 100
dataX = []
dataY = []
for i in range(0, n_chars - seq_length, 1):
    seq_in = raw_text[i:i + seq_length]
    seq_out = raw_text[i + seq_length]
    dataX.append([char_to_int[char] for char in seq_in])
    dataY.append(char_to_int[seq_out])
n_patterns = len(dataX)
    print("Total Patterns: ", n_patterns)
```

Listing 35.5: Create input/output patterns from raw dataset

Running the code to this point shows that when you split up the dataset into training data for the network to learn that you have just under 150,000 training patterns. This makes sense as, excluding the first 100 characters, you have one training pattern to predict each of the remaining characters.

```
Total Patterns:  147574
```

Output 35.6: Output summary of created patterns

Now that you have prepared your training data, you need to transform it to be suitable for use with Keras. First, you must transform the list of input sequences into the form [*samples, time steps, features*] expected by an LSTM network.

Next, you need to rescale the integers to the range 0-to-1 to make the patterns easier to learn by the LSTM network using the sigmoid activation function by default.

Finally, you need to convert the output patterns (single characters converted to integers) into a one-hot encoding. This is so that you can configure the network to predict the probability of each of the 47 different characters in the vocabulary (an easier representation) rather than trying to force it to predict precisely the next character. Each y value is converted into a sparse vector with a length of 47, full of zeros, except with a 1 in the column for the letter (integer) that the pattern represents. For example, when "n" (integer value 31) is one-hot encoded it looks as follows:

```
[ 0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.
  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  1.  0.  0.  0.  0.
  0.  0.  0.  0.  0.  0.  0.  0.]
```

Output 35.7: Sample of one-hot encoded output integer

You can implement these steps as below.

```
...
# reshape X to be [samples, time steps, features]
X = np.reshape(dataX, (n_patterns, seq_length, 1))
# normalize
X = X / float(n_vocab)
# one-hot encode the output variable
y = to_categorical(dataY)
```

Listing 35.6: Prepare data ready for modeling

You can now define your LSTM model. Here, you define a single hidden `LSTM` layer with 256 memory units. The network uses dropout with a probability of 20. The output layer is a `Dense` layer using the `softmax` activation function to output a probability prediction for each of the 47 characters between 0 and 1. The problem is really a single character classification problem with 47 classes and, as such, is defined as optimizing the log loss (cross-entropy) using the Adam optimization algorithm for speed.

```
...
# define the LSTM model
model = Sequential()
model.add(LSTM(256, input_shape=(X.shape[1], X.shape[2])))
model.add(Dropout(0.2))
model.add(Dense(y.shape[1], activation='softmax'))
model.compile(loss='categorical_crossentropy', optimizer='adam')
```

Listing 35.7: Create LSTM network to model the dataset

There is no test dataset. You are modeling the entire training dataset to learn the probability of each character in a sequence. You are not interested in the most accurate (classification accuracy) model of the training dataset. This would not be a model that predicts each character in the training dataset perfectly. Instead, you are interested in a generalization of the dataset that minimizes the chosen loss function. You are seeking a balance between generalization and overfitting but short of memorization.

The network is slow to train (about 300 seconds per epoch on an Nvidia K520 GPU). Because of the slowness and because of the optimization requirements, use model checkpointing

to record all the network weights to file each time an improvement in loss is observed at the end of the epoch. You will use the best set of weights (lowest loss) to instantiate your generative model in the next section.

```
...
# define the checkpoint
filepath="weights-improvement-{epoch:02d}-{loss:.4f}.hdf5"
checkpoint = ModelCheckpoint(filepath, monitor='loss',
                             verbose=1, save_best_only=True, mode='min')
callbacks_list = [checkpoint]
```

Listing 35.8: Create checkpoints for best seen model

You can now fit your model to the data. Here, you use a modest number of 20 epochs and a large batch size of 128 patterns.

```
model.fit(X, y, epochs=20, batch_size=128, callbacks=callbacks_list)
```

Listing 35.9: Fit the model

The full code listing is provided below for completeness.

```
# Small LSTM Network to Generate Text for Alice in Wonderland
import numpy as np
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.layers import Dropout
from tensorflow.keras.layers import LSTM
from tensorflow.keras.callbacks import ModelCheckpoint
from tensorflow.keras.utils import to_categorical
# load ascii text and covert to lowercase
filename = "wonderland.txt"
raw_text = open(filename, 'r', encoding='utf-8').read()
raw_text = raw_text.lower()
# create mapping of unique chars to integers
chars = sorted(list(set(raw_text)))
char_to_int = dict((c, i) for i, c in enumerate(chars))
# summarize the loaded data
n_chars = len(raw_text)
n_vocab = len(chars)
print("Total Characters: ", n_chars)
print("Total Vocab: ", n_vocab)
# prepare the dataset of input to output pairs encoded as integers
seq_length = 100
dataX = []
dataY = []
for i in range(0, n_chars - seq_length, 1):
    seq_in = raw_text[i:i + seq_length]
    seq_out = raw_text[i + seq_length]
    dataX.append([char_to_int[char] for char in seq_in])
    dataY.append(char_to_int[seq_out])
n_patterns = len(dataX)
print("Total Patterns: ", n_patterns)
```

```
# reshape X to be [samples, time steps, features]
X = np.reshape(dataX, (n_patterns, seq_length, 1))
# normalize
X = X / float(n_vocab)
# one-hot encode the output variable
y = to_categorical(dataY)
# define the LSTM model
model = Sequential()
model.add(LSTM(256, input_shape=(X.shape[1], X.shape[2])))
model.add(Dropout(0.2))
model.add(Dense(y.shape[1], activation='softmax'))
model.compile(loss='categorical_crossentropy', optimizer='adam')
# define the checkpoint
filepath="weights-improvement-{epoch:02d}-{loss:.4f}.hdf5"
checkpoint = ModelCheckpoint(filepath, monitor='loss',
                             verbose=1, save_best_only=True, mode='min')
callbacks_list = [checkpoint]
# fit the model
model.fit(X, y, epochs=20, batch_size=128, callbacks=callbacks_list)
```

Listing 35.10: Complete code listing for LSTM to model dataset

Note: Your results may vary given the stochastic nature of the algorithm or evaluation procedure, or differences in numerical precision. Consider running the example a few times and compare the average outcome.

After running the example, you should have a number of weight checkpoint files in the local directory. You can delete them all except the one with the smallest loss value. For example, when this example was run, you can see below the checkpoint with the smallest loss that was achieved.

```
weights-improvement-19-1.9435.hdf5
```

Output 35.8: Sample of checkpoint weights for well performing model

The network loss decreased almost every epoch, so the network could likely benefit from training for many more epochs. In the next section, you will look at using this model to generate new text sequences.

35.3 Generating Text with an LSTM Network

Generating text using the trained LSTM network is relatively straightforward. Firstly, you will load the data and define the network in exactly the same way, except the network weights are loaded from a checkpoint file, and the network does not need to be trained.

Note: It seems that you might need to use the same machine/environment to generate text as was used to create the network weights (e.g., GPUs or CPUs), otherwise the network might just generate garbage.

```
...
# load the network weights
filename = "weights-improvement-19-1.9435.hdf5"
model.load_weights(filename)
model.compile(loss='categorical_crossentropy', optimizer='adam')
```

Listing 35.11: Load checkpoint network weights

Also, when preparing the mapping of unique characters to integers, you must also create a reverse mapping that you can use to convert the integers back to characters so that you can understand the predictions.

```
...
int_to_char = dict((i, c) for i, c in enumerate(chars))
```

Listing 35.12: Mapping from integers to characters

Finally, you need to actually make predictions. The simplest way to use the Keras LSTM model to make predictions is to first start with a seed sequence as input, generate the next character, then update the seed sequence to add the generated character on the end and trim off the first character. This process is repeated for as long as you want to predict new characters (e.g., a sequence of 1,000 characters in length). You can pick a random input pattern as your seed sequence, then print generated characters as you generate them.

```
...
# pick a random seed
start = np.random.randint(0, len(dataX)-1)
pattern = dataX[start]
print("Seed:")
print("\"", ''.join([int_to_char[value] for value in pattern]), "\"")
# generate characters
for i in range(1000):
    x = np.reshape(pattern, (1, len(pattern), 1))
    x = x / float(n_vocab)
    prediction = model.predict(x, verbose=0)
    index = np.argmax(prediction)
    result = int_to_char[index]
    seq_in = [int_to_char[value] for value in pattern]
    sys.stdout.write(result)
    pattern.append(index)
    pattern = pattern[1:len(pattern)]
print("\nDone.")
```

Listing 35.13: Seed network and generate text

The full code example for generating text using the loaded LSTM model is listed below for completeness.

```
# Load LSTM network and generate text
import sys
import numpy as np
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
```

```
from tensorflow.keras.layers import Dropout
from tensorflow.keras.layers import LSTM
from tensorflow.keras.callbacks import ModelCheckpoint
from tensorflow.keras.utils import to_categorical
# load ascii text and covert to lowercase
filename = "wonderland.txt"
raw_text = open(filename, 'r', encoding='utf-8').read()
raw_text = raw_text.lower()
# create mapping of unique chars to integers, and a reverse mapping
chars = sorted(list(set(raw_text)))
char_to_int = dict((c, i) for i, c in enumerate(chars))
int_to_char = dict((i, c) for i, c in enumerate(chars))
# summarize the loaded data
n_chars = len(raw_text)
n_vocab = len(chars)
print("Total Characters: ", n_chars)
print("Total Vocab: ", n_vocab)
# prepare the dataset of input to output pairs encoded as integers
seq_length = 100
dataX = []
dataY = []
for i in range(0, n_chars - seq_length, 1):
    seq_in = raw_text[i:i + seq_length]
    seq_out = raw_text[i + seq_length]
    dataX.append([char_to_int[char] for char in seq_in])
    dataY.append(char_to_int[seq_out])
n_patterns = len(dataX)
print("Total Patterns: ", n_patterns)
# reshape X to be [samples, time steps, features]
X = np.reshape(dataX, (n_patterns, seq_length, 1))
# normalize
X = X / float(n_vocab)
# one-hot encode the output variable
y = to_categorical(dataY)
# define the LSTM model
model = Sequential()
model.add(LSTM(256, input_shape=(X.shape[1], X.shape[2])))
model.add(Dropout(0.2))
model.add(Dense(y.shape[1], activation='softmax'))
# load the network weights (modify to your filename)
filename = "weights-improvement-19-1.9435.hdf5"
model.load_weights(filename)
model.compile(loss='categorical_crossentropy', optimizer='adam')
# pick a random seed
start = np.random.randint(0, len(dataX)-1)
pattern = dataX[start]
print("Seed:")
print("\"", ''.join([int_to_char[value] for value in pattern]), "\"")
# generate characters
for i in range(1000):
    x = np.reshape(pattern, (1, len(pattern), 1))
    x = x / float(n_vocab)
    prediction = model.predict(x, verbose=0)
    index = np.argmax(prediction)
```

```
    result = int_to_char[index]
    seq_in = [int_to_char[value] for value in pattern]
    sys.stdout.write(result)
    pattern.append(index)
    pattern = pattern[1:len(pattern)]
print("\nDone.")
```

Listing 35.14: Complete code listing for generating text for the small LSTM network

Running this example first outputs the selected random seed, then each character as it is generated. For example, below are the results from one run of this text generator. The random seed was:

```
be no mistake about it: it was neither more nor less than a pig, and she
felt that it would be quit
```

Output 35.9: Randomly selected sequence used to seed the network

The generated text with the random seed (cleaned up for presentation) was as follows.

```
be no mistake about it: it was neither more nor less than a pig, and she
felt that it would be quit e aelin that she was a little want oe toiet
ano a grtpersent to the tas a little war th tee the tase oa teettee
the had been tinhgtt a little toiee at the cadl in a long tuiee aedun
thet sheer was a little tare gereen to be a gentle of the tabdit  soenee
the gad  ouw ie the tay a tirt of toiet at the was a little
anonersen, and thiu had been woite io a lott of tueh a tiie  and taede
bot her aeain  she cere thth the bene tith the tere bane to tee
toaete to tee the harter was a little tire the same oare cade an anl ano
the garee and the was so seat the was a little gareen and the sabdit,
and the white rabbit wese tilel an the caoe and the sabbit se teeteer,
and the white rabbit wese tilel an the cade in a lonk tfne the sabdi
ano aroing to tea the was sf teet whitg the was a little tane oo thete
the sabeit  she was a little tartig to the tar tf tee the tame of the
cagd, and the white rabbit was a little toiee to be anle tite thete ofs
and the tabdit was the wiite rabbit, and
```

Output 35.10: Generated text with random seed text

Let's note some observations about the generated text.

- ▷ It generally conforms to the line format observed in the original text of fewer than 80 characters before a new line.
- ▷ The characters are separated into word-like groups, and most groups are actual English words (e.g., "the," "little," and "was"), but many are not (e.g., "lott," "tiie," and "taede").
- ▷ Some of the words in sequence make sense (e.g., "*and the white rabbit*"), but many do not (e.g., "*wese tilel*").

The fact that this character-based model of the book produces output like this is very impressive. It gives you a sense of the learning capabilities of LSTM networks. However, the results are not perfect. In the next section, you will look at improving the quality of results by developing a much larger LSTM network.

35.4 Larger LSTM Recurrent Neural Network

You got results, but not excellent results in the previous section. Now, you can try to improve the quality of the generated text by creating a much larger network. You will keep the number of memory units the same at 256 but add a second layer.

```
...
model = Sequential()
model.add(LSTM(256, input_shape=(X.shape[1], X.shape[2]), return_sequences=True))
model.add(Dropout(0.2))
model.add(LSTM(256))
model.add(Dropout(0.2))
model.add(Dense(y.shape[1], activation='softmax'))
model.compile(loss='categorical_crossentropy', optimizer='adam')
```

Listing 35.15: Define a stacked LSTM network

You will also change the filename of the checkpointed weights so that you can tell the difference between weights for this network and the previous (by appending the word "`bigger`" in the filename).

```
filepath="weights-improvement-{epoch:02d}-{loss:.4f}-bigger.hdf5"
```

Listing 35.16: Filename for checkpointing network weights for larger model

Finally, you will increase the number of training epochs from 20 to 50 and decrease the batch size from 128 to 64 to give the network more of an opportunity to be updated and learn. The full code listing is presented below for completeness.

```
# Larger LSTM Network to Generate Text for Alice in Wonderland
import numpy as np
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.layers import Dropout
from tensorflow.keras.layers import LSTM
from tensorflow.keras.callbacks import ModelCheckpoint
from tensorflow.keras.utils import to_categorical
# load ascii text and covert to lowercase
filename = "wonderland.txt"
raw_text = open(filename, 'r', encoding='utf-8').read()
raw_text = raw_text.lower()
# create mapping of unique chars to integers
chars = sorted(list(set(raw_text)))
char_to_int = dict((c, i) for i, c in enumerate(chars))
# summarize the loaded data
n_chars = len(raw_text)
n_vocab = len(chars)
print("Total Characters: ", n_chars)
print("Total Vocab: ", n_vocab)
# prepare the dataset of input to output pairs encoded as integers
seq_length = 100
dataX = []
```

```
dataY = []
for i in range(0, n_chars - seq_length, 1):
    seq_in = raw_text[i:i + seq_length]
    seq_out = raw_text[i + seq_length]
    dataX.append([char_to_int[char] for char in seq_in])
    dataY.append(char_to_int[seq_out])
n_patterns = len(dataX)
print("Total Patterns: ", n_patterns)
# reshape X to be [samples, time steps, features]
X = np.reshape(dataX, (n_patterns, seq_length, 1))
# normalize
X = X / float(n_vocab)
# one-hot encode the output variable
y = to_categorical(dataY)
# define the LSTM model
model = Sequential()
model.add(LSTM(256, input_shape=(X.shape[1], X.shape[2]), return_sequences=True))
model.add(Dropout(0.2))
model.add(LSTM(256))
model.add(Dropout(0.2))
model.add(Dense(y.shape[1], activation='softmax'))
model.compile(loss='categorical_crossentropy', optimizer='adam')
# define the checkpoint
filepath = "weights-improvement-{epoch:02d}-{loss:.4f}-bigger.hdf5"
checkpoint = ModelCheckpoint(filepath, monitor='loss',
                             verbose=1, save_best_only=True, mode='min')
callbacks_list = [checkpoint]
# fit the model
model.fit(X, y, epochs=50, batch_size=64, callbacks=callbacks_list)
```

Listing 35.17: Complete code listing for the larger LSTM network

Running this example takes some time, at least 700 seconds per epoch. After running this example, you may achieve a loss of about 1.2. For example, the best result I achieved from running this model was stored in a checkpoint file with the name:

```
weights-improvement-47-1.2219-bigger.hdf5
```

Output 35.11: Sample of checkpoint weights for well performing larger model

This achieved a loss of 1.2219 at epoch 47. As in the previous section, you can use this best model from the run to generate text. The only change you need to make to the text generation script from the previous section is in the specification of the network topology and from which file to seed the network weights. The full code listing is provided below for completeness.

```
# Load Larger LSTM network and generate text
import sys
import numpy as np
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.layers import Dropout
from tensorflow.keras.layers import LSTM
from tensorflow.keras.callbacks import ModelCheckpoint
```

```
from tensorflow.keras.utils import to_categorical
# load ascii text and covert to lowercase
filename = "wonderland.txt"
raw_text = open(filename, 'r', encoding='utf-8').read()
raw_text = raw_text.lower()
# create mapping of unique chars to integers, and a reverse mapping
chars = sorted(list(set(raw_text)))
char_to_int = dict((c, i) for i, c in enumerate(chars))
int_to_char = dict((i, c) for i, c in enumerate(chars))
# summarize the loaded data
n_chars = len(raw_text)
n_vocab = len(chars)
print("Total Characters: ", n_chars)
print("Total Vocab: ", n_vocab)
# prepare the dataset of input to output pairs encoded as integers
seq_length = 100
dataX = []
dataY = []
for i in range(0, n_chars - seq_length, 1):
    seq_in = raw_text[i:i + seq_length]
    seq_out = raw_text[i + seq_length]
    dataX.append([char_to_int[char] for char in seq_in])
    dataY.append(char_to_int[seq_out])
n_patterns = len(dataX)
print("Total Patterns: ", n_patterns)
# reshape X to be [samples, time steps, features]
X = np.reshape(dataX, (n_patterns, seq_length, 1))
# normalize
X = X / float(n_vocab)
# one-hot encode the output variable
y = to_categorical(dataY)
# define the LSTM model
model = Sequential()
model.add(LSTM(256, input_shape=(X.shape[1], X.shape[2]), return_sequences=True))
model.add(Dropout(0.2))
model.add(LSTM(256))
model.add(Dropout(0.2))
model.add(Dense(y.shape[1], activation='softmax'))
# load the network weights (modify to your filename)
filename = "weights-improvement-47-1.2219-bigger.hdf5"
model.load_weights(filename)
model.compile(loss='categorical_crossentropy', optimizer='adam')
# pick a random seed
start = np.random.randint(0, len(dataX)-1)
pattern = dataX[start]
print("Seed:")
print("\"", ''.join([int_to_char[value] for value in pattern]), "\"")
# generate characters
for i in range(1000):
    x = np.reshape(pattern, (1, len(pattern), 1))
    x = x / float(n_vocab)
    prediction = model.predict(x, verbose=0)
    index = np.argmax(prediction)
    result = int_to_char[index]
```

```
    seq_in = [int_to_char[value] for value in pattern]
    sys.stdout.write(result)
    pattern.append(index)
    pattern = pattern[1:len(pattern)]
print("\nDone.")
```

Listing 35.18: Complete code listing for generating text with the larger LSTM network

One example of running this text generation script produces the output below. The randomly chosen seed text was:

```
d herself lying on the bank, with her
head in the lap of her sister, who was gently brushing away s
```

Output 35.12: Randomly selected sequence used to seed the network

The generated text with the seed (cleaned up for presentation) was :

```
herself lying on the bank, with her
head in the lap of her sister, who was gently brushing away
so siee, and she sabbit said to herself and the sabbit said to herself and the sood
way of the was a little that she was a little lad good to the garden,
and the sood of the mock turtle said to herself, 'it was a little that
the mock turtle said to see it said to sea it said to sea it say it
the marge hard sat hn a little that she was so sereated to herself, and
she sabbit said to herself, 'it was a little little shated of the sooe
of the coomouse it was a little lad good to the little gooder head. and
said to herself, 'it was a little little shated of the mouse of the
good of the courte, and it was a little little shated in a little that
the was a little little shated of the thmee said to see it was a little
book of the was a little that she was so sereated to hare a little the
began sitee of the was of the was a little that she was so seally and
the sabbit was a little lad good to the little gooder head of the gad
seared to see it was a little lad good to the little good
```

Output 35.13: Generated text with random seed text

You can see that there are generally fewer spelling mistakes, and the text looks more realistic but is still quite nonsensical. For example, the same phrases get repeated again and again, like "*said to herself*" and "*little*." Quotes are opened but not closed. These are better results, but there is still a lot of room for improvement.

35.5 Extension Ideas to Improve the Model

Below are some ideas that you may want to investigate to further improve the model:

- ▷ Predict fewer than 1,000 characters as output for a given seed
- ▷ Remove all punctuation from the source text and, therefore, from the models' vocabulary
- ▷ Try one-hot encoding for the input sequences

- ▷ Train the model on padded sentences rather than random sequences of characters
- ▷ Increase the number of training epochs to 100 or many hundreds
- ▷ Add dropout to the visible input layer and consider tuning the dropout percentage
- ▷ Tune the batch size; try a batch size of 1 as a (very slow) baseline and larger sizes from there
- ▷ Add more memory units to the layers and/or more layers
- ▷ Experiment with scale factors (temperature) when interpreting the prediction probabilities
- ▷ Change the LSTM layers to be *stateful* to maintain state across batches

35.6 Further Reading

This character text model is a popular way of generating text using recurrent neural networks. Below are some more resources and tutorials on the topic if you are interested in going deeper. Perhaps the most popular is the tutorial by Karpathy, *The Unreasonable Effectiveness of Recurrent Neural Networks*.

Papers

Ilya Sutskever, James Martens, and Geoffrey Hinton. "Generating Text with Recurrent Neural Networks". In: *Proceedings of the 28th International Conference on Machine Learning*. Bellevue, WA, USA, 2011.
https://www.cs.utoronto.ca/~ilya/pubs/2011/LANG-RNN.pdf

Articles

Andrej Karpathy. *The Unreasonable Effectiveness of Recurrent Neural Networks*. May 2015.
http://karpathy.github.io/2015/05/21/rnn-effectiveness/

François Chollet. *Character-level text generation with LSTM*. Keras code examples, 2020.
https://keras.io/examples/generative/lstm_character_level_text_generation/

Char RNN Example. MXNet documentation.
https://mxnet.apache.org/versions/0.11.0/tutorials/r/charRnnModel.html

Lars Eidnes. *Auto-Generating Clickbait With Recurrent Neural Networks*. 2015.
https://larseidnes.com/2015/10/13/auto-generating-clickbait-with-recurrent-neural-networks/

35.7 Summary

In this chapter, you discovered how you can develop an LSTM recurrent neural network for text generation in Python with the Keras deep learning library. After completing this chapter, you know:

- ▷ Where to download the ASCII text for classical books for free that you can use for training

- ▷ How to train an LSTM network on text sequences and how to use the trained network to generate new sequences
- ▷ How to develop stacked LSTM networks and lift the performance of the model

This marks the end of this book.